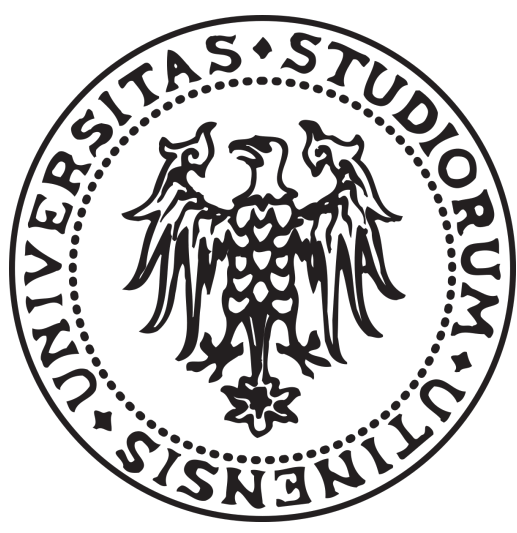




INGEGNERIA DEL SOFTWARE – 2025–26

DIAGRAMMA DI SEQUENZA

LEZIONE 17
24/11/2025
VINCENZO RICCIO

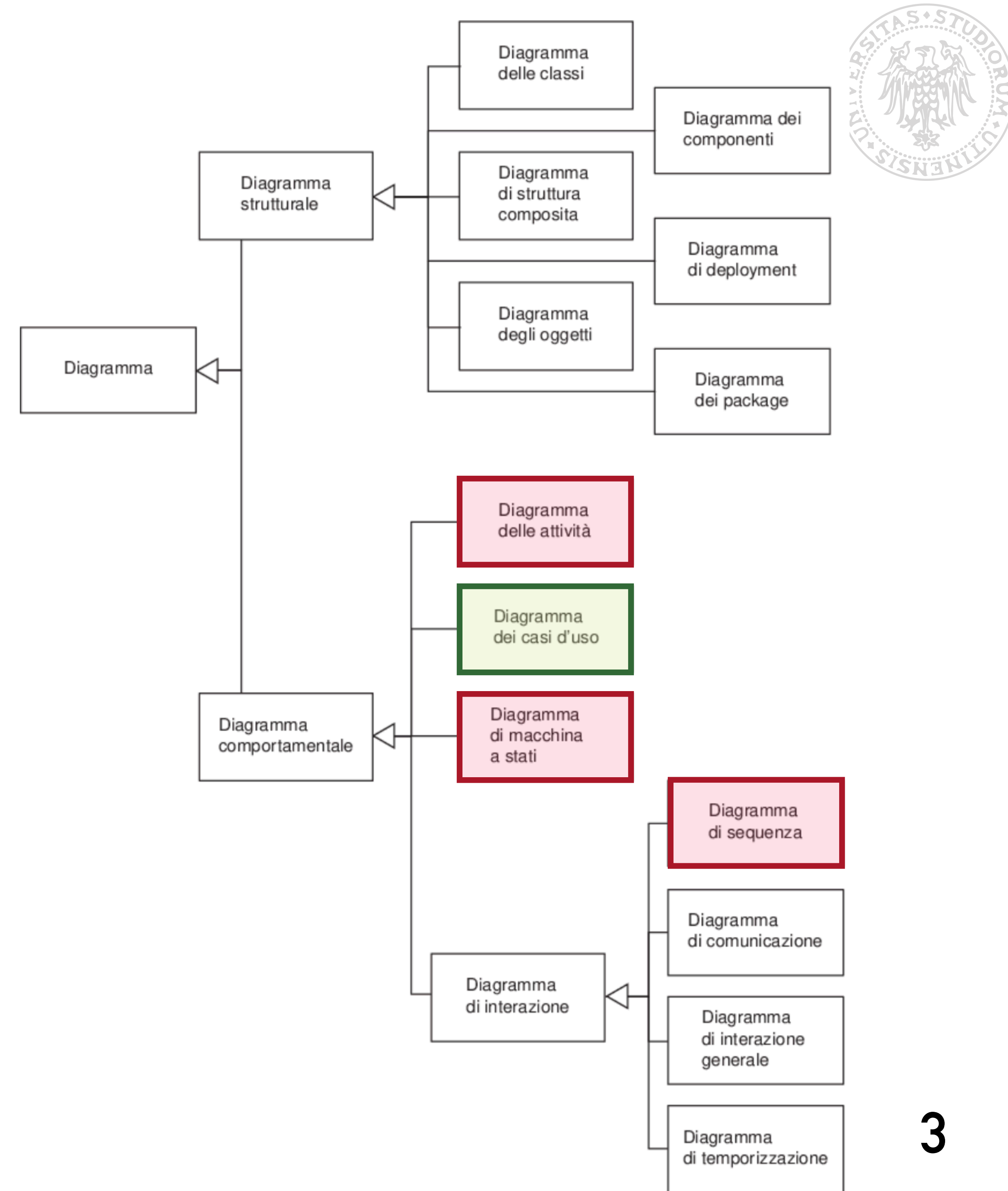


- Fowler - Capitolo 4



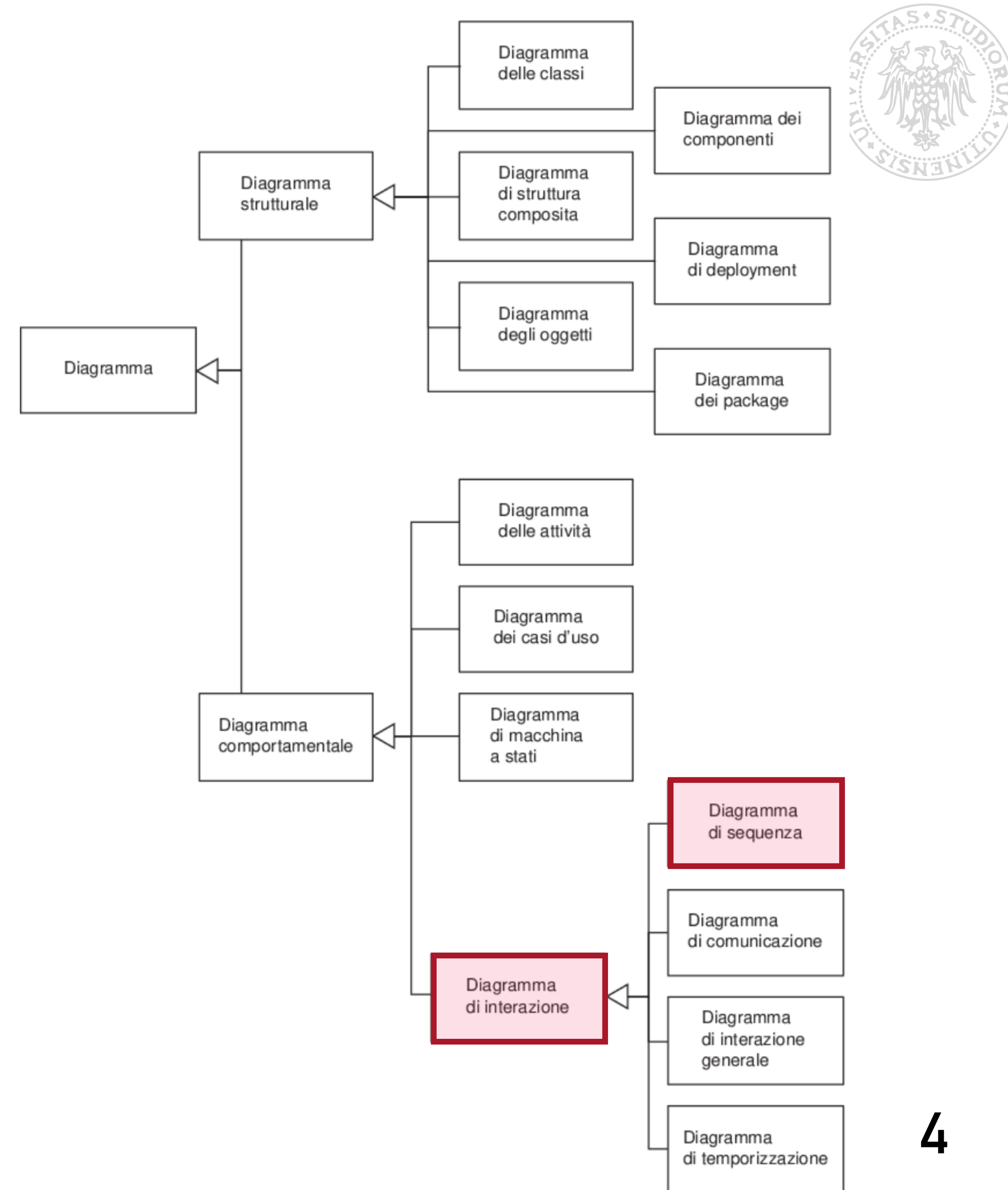
DIAGRAMMI UML COMPORTAMENTALI

- Diagrammi che modellano il comportamento dinamico degli elementi che compongono un sistema (descritti, a loro volta, nei diagrammi strutturali)
- Oltre al diagramma dei casi d'uso, in questo corso studieremo:
 - Activity Diagram
 - Statechart Diagram
 - Sequence Diagram



DIAGRAMMI DI INTERAZIONE

- I Diagrammi di Interazione modellano la **collaborazione dinamica** tra oggetti che implementano collettivamente un determinato comportamento
- I diagrammi di interazione più utilizzati sono i diagrammi di sequenza



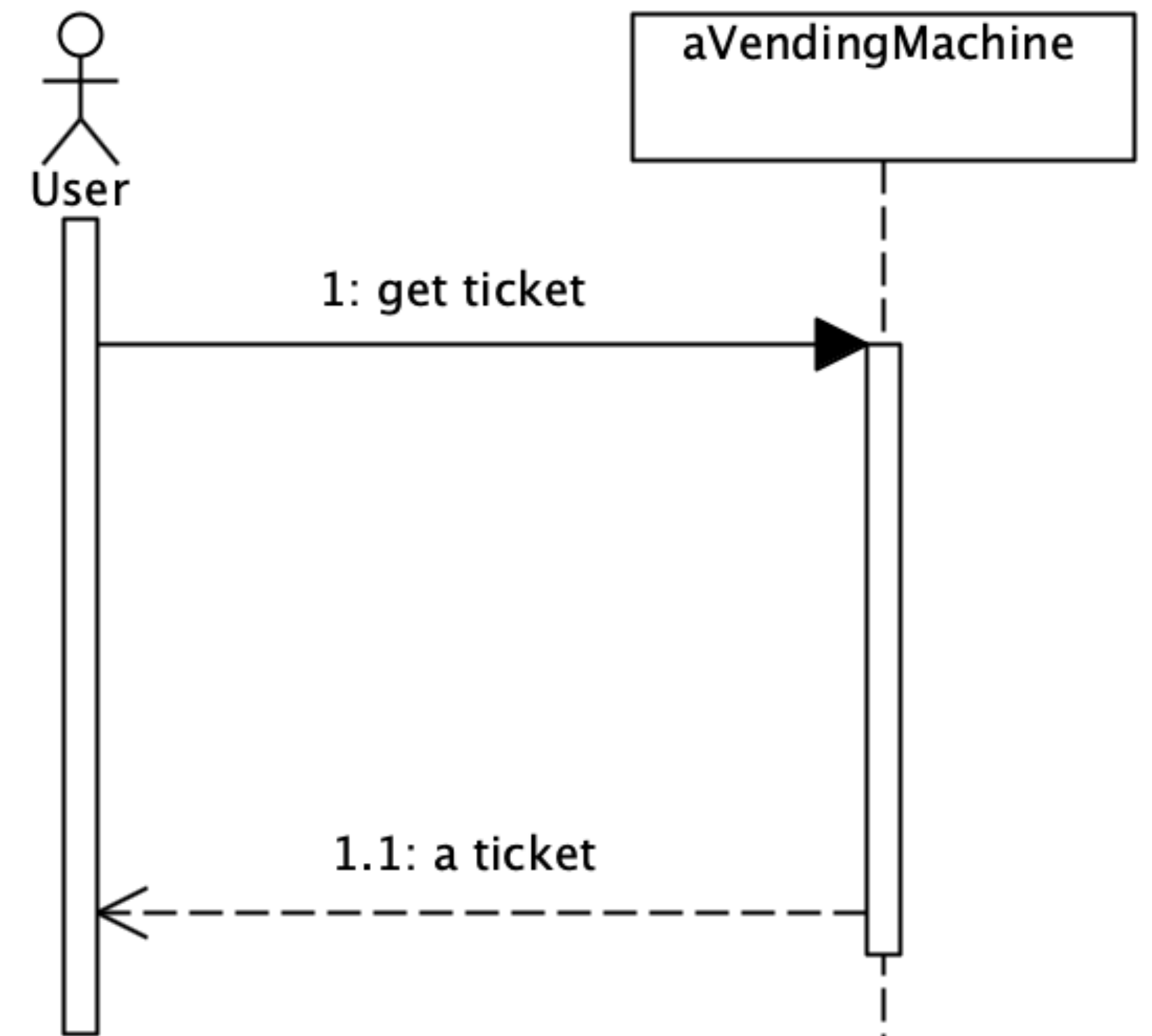


DIAGRAMMI DI SEQUENZA: NOTAZIONE

DIAGRAMMI DI SEQUENZA



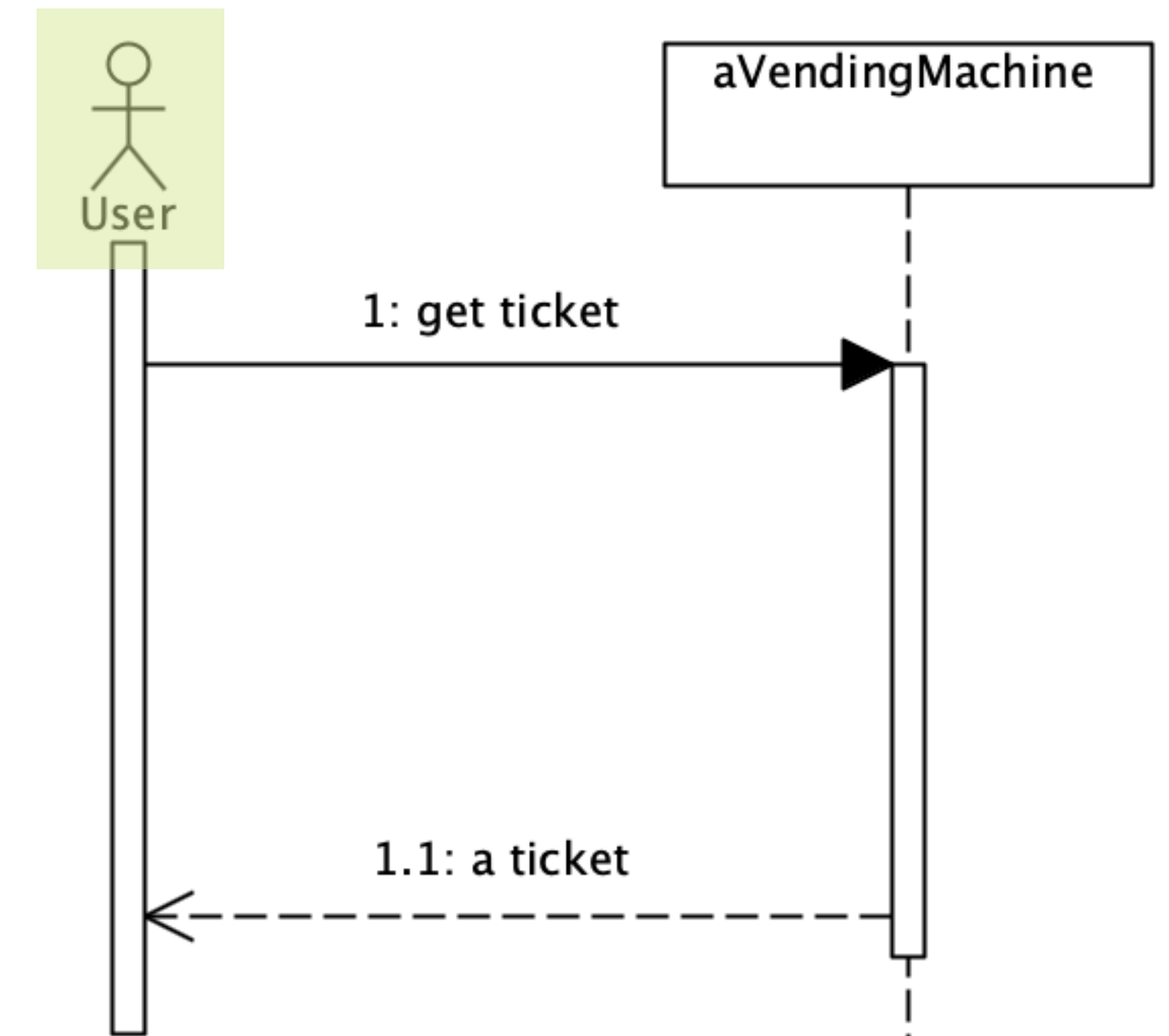
- Modella le interazioni tra uno o più attori e le parti di un sistema software, nell'ambito dell'esecuzione di uno o più scenari di un caso d'uso
- Le interazioni tra **attori** e gli **oggetti** del sistema sono modellate in forma di **messaggi**, così come prevede il paradigma orientato agli oggetti
- Lo scambio di messaggi è letto dall'alto in basso
- Chiariscono le chiamate tra i partecipanti e le responsabilità di ciascuno di essi



ATTORE



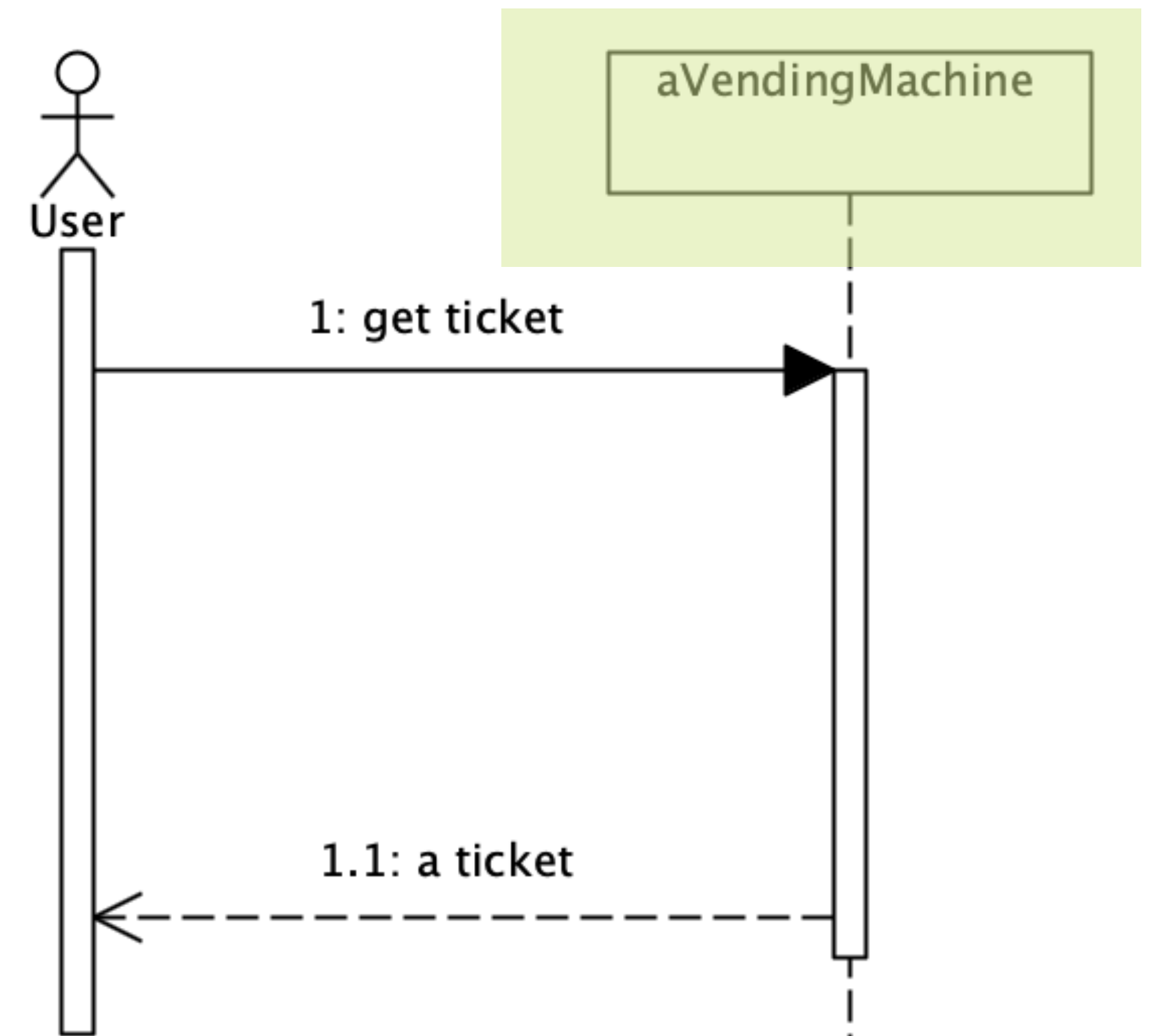
- Gli attori sono rappresentati come nel diagramma dei casi d'uso
- Riportati sulla sinistra, con frecce che modellano le interazioni verso gli oggetti del sistema (ossia i messaggi scambiati)
- Possono non essere presenti se lo scenario é avviato da un altro scenario o iniziato dal sistema



PARTECIPANTE O ISTANZA DI CLASSE



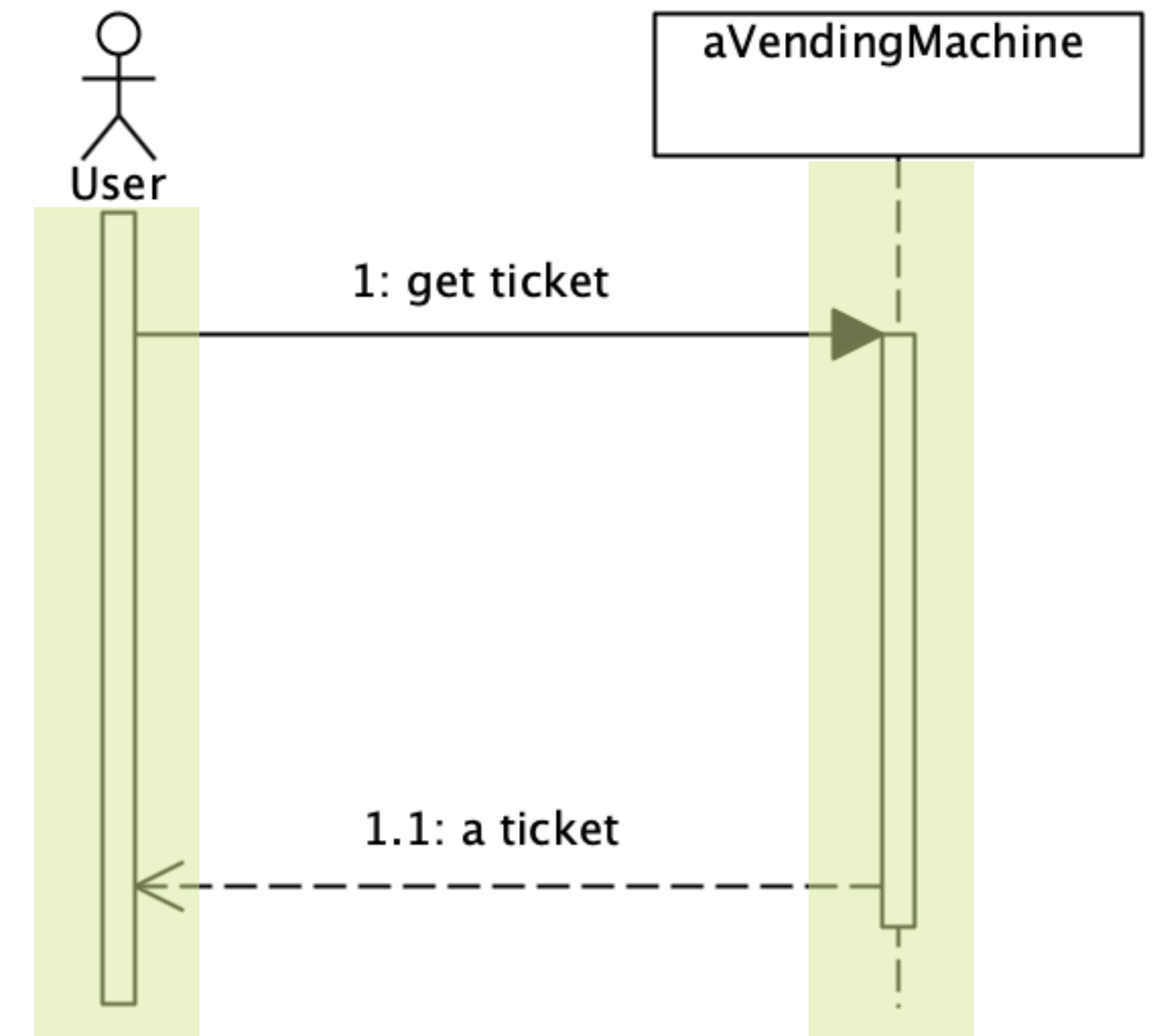
- Le istanze delle classi sono rappresentate da rettangoli etichettati in uno dei seguenti modi:
 - Con il nome della classe e l'identificatore dell'oggetto nel formato **nomeOggetto:nomeClasse**
 - Più informalmente con un nome che suggerisce che si sta considerando un'istanza di una classe (anOrder, aProduct, ...)



LIFELINE



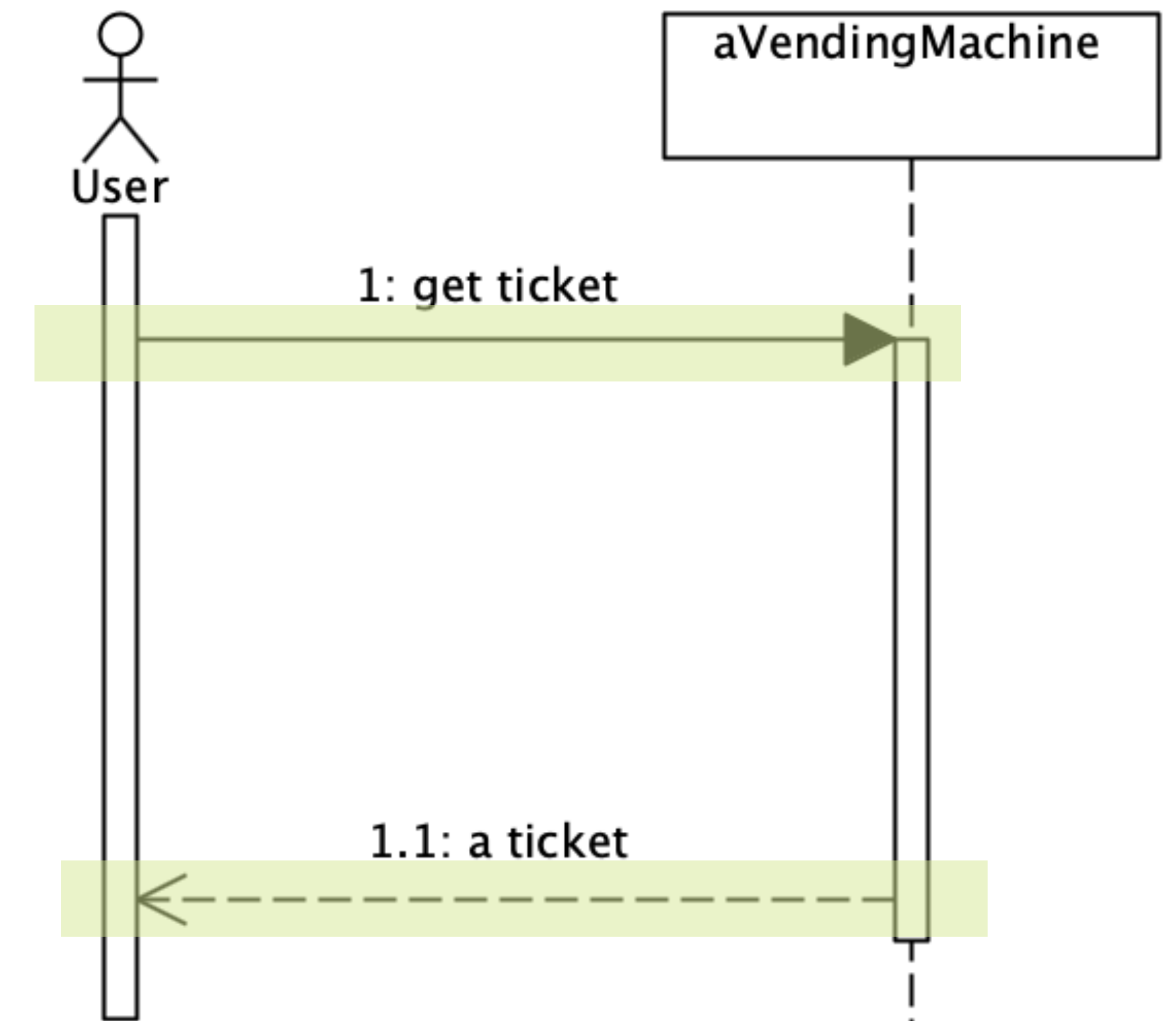
- Le lifeline (linee di vita) mostrano i partecipanti attraverso una linea verticale tratteggiata dall'alto in basso che parte dal rettangolo o dal simbolo di attore che rappresenta il partecipante
- Rappresentano il periodo temporale di vita dell'oggetto
- La lifeline mostra l'ordinamento dei messaggi, non la durata temporale dell'interazione



MESSAGGIO



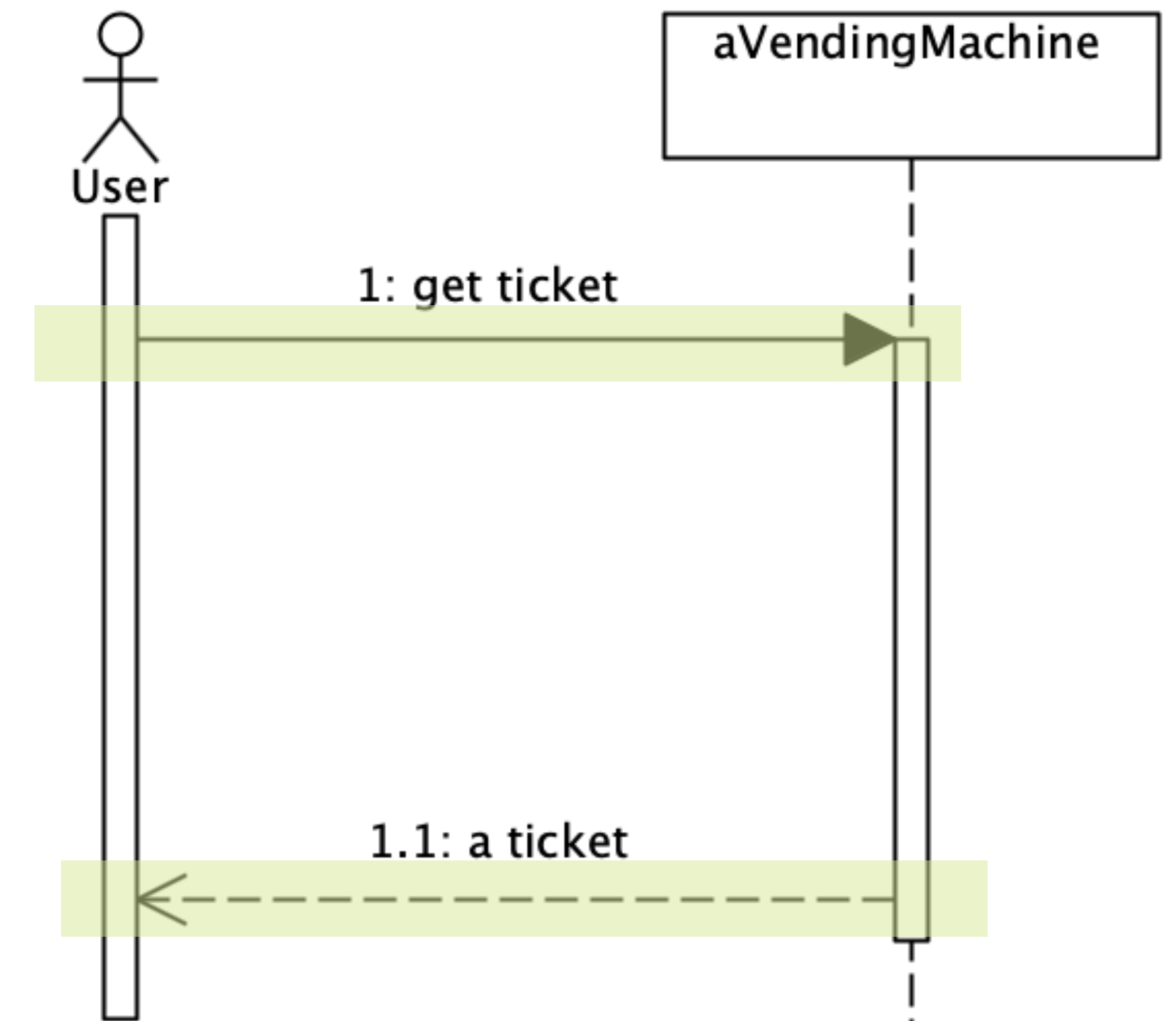
- Un messaggio definisce una particolare comunicazione tra le lifeline in una interazione
- Rappresentati come frecce tra lifeline (tra un attore e un oggetto oppure tra oggetti)
- L'ordine dei messaggi (dall'alto verso il basso) ricalca l'ordine sequenziale con il quale essi vengono scambiati



SCAMBIO DI MESSAGGI



- Un messaggio può:
 - ⊙ Invocare una operazione (messaggio di **chiamata**)
 - ⊙ Creare o distruggere una istanza (messaggio di creazione/distruzione)
 - ⊙ Inviare un segnale o un dato
- Quando una lifeline riceve un messaggio di chiamata, deve esistere una **corrispondente operazione nella classe della lifeline ricevente**
- I messaggi possono invocare operazioni in maniera sincrona o asincrona



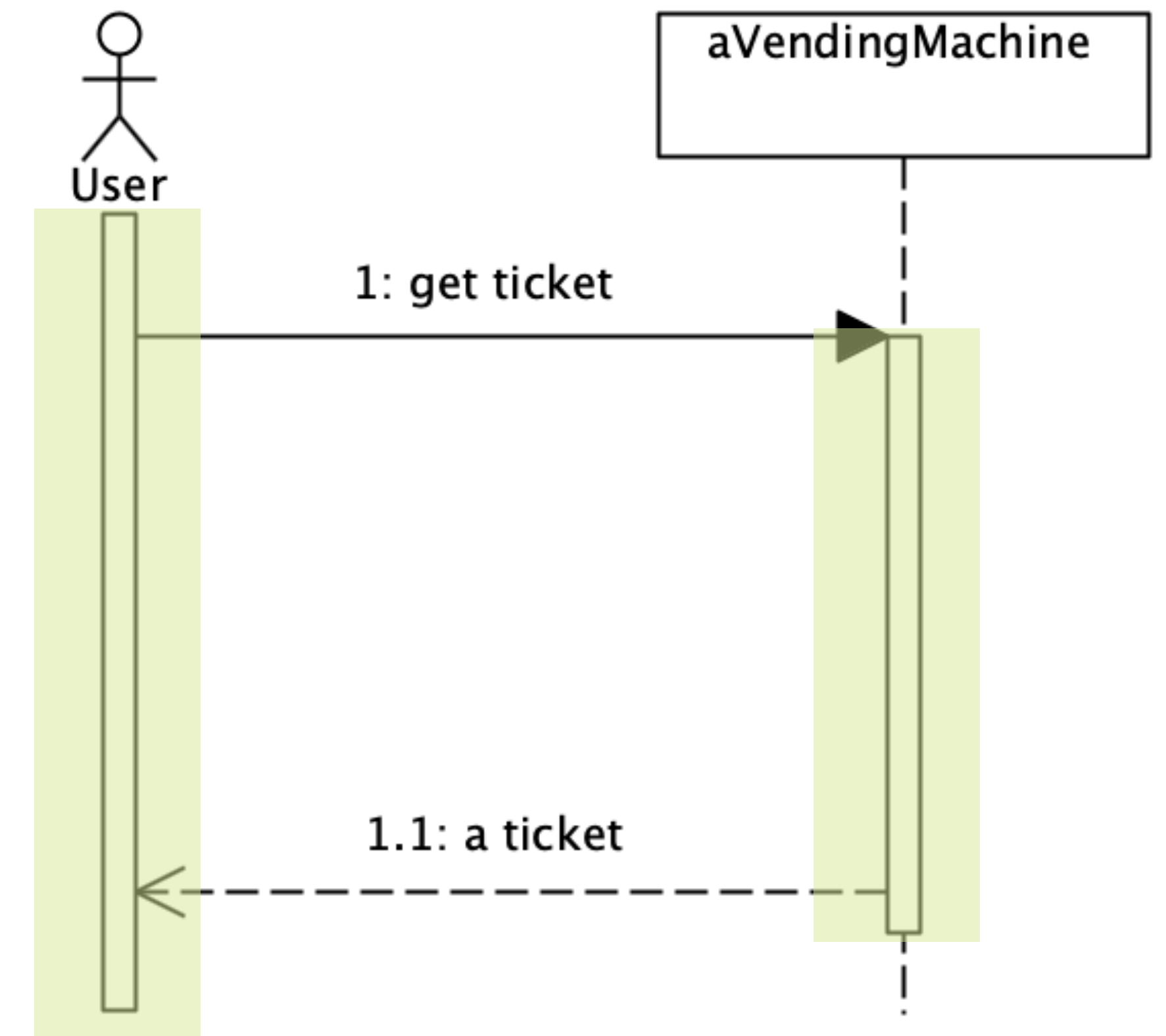


Rappresentazione Sender Receiver		Tipo	Semantica
		Messaggio Sincrono	Il chiamante attende che il ricevente completi
		Messaggio Asincrono	Il chiamante invia un segnale al ricevente e non attende il completamento
		Messaggio di ritorno	Il ricevente completa una chiamata sincrona e restituisce il controllo al chiamante
		Creazione	Il chiamante crea il target (invocando, ad esempio, un costruttore)
		Distruzione	Il chiamante distrugge il ricevente
		Messaggio trovato	Il messaggio è inviato esternamente all'ambito dell'interazione
		Messaggio perso	Il messaggio non raggiunge la destinazione (può modellare i casi di errore)

ACTIVATION BOX



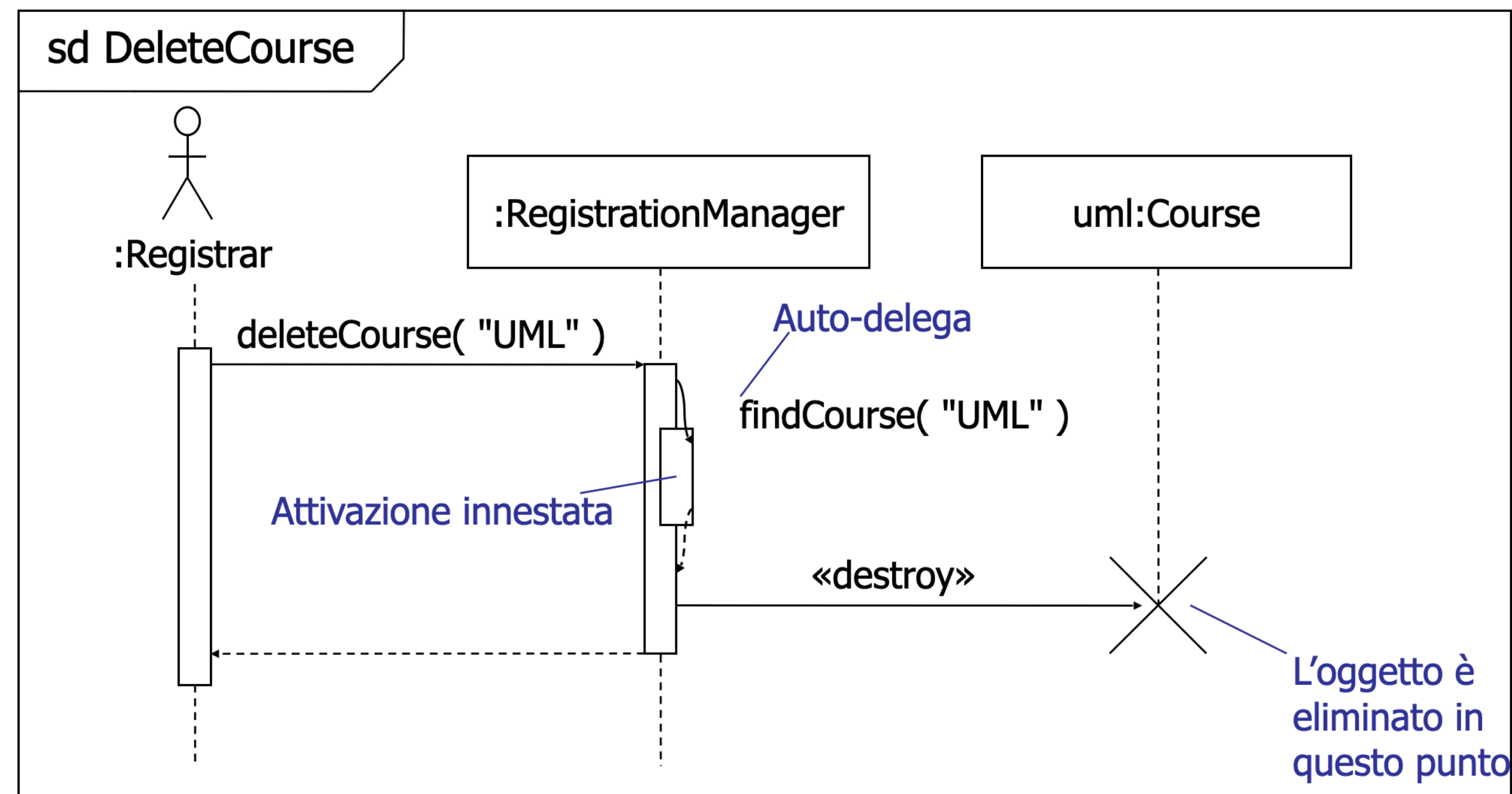
- Le activation box o barre di attivazione sono rettangoli che coprono parte delle lifeline
- Rappresentano il periodo di tempo in cui l'istanza é attiva nell'interazione (ad es. tempo necessario ad un oggetto per elaborare un messaggio di richiesta)
- Una lifeline di un partecipante si attiva quando esso elabora un messaggio: durante l'esecuzione, col procedere del tempo, l'attivazione si sposta tra le lifeline descrivendo il flusso di controllo



MESSAGGI DI AUTO-DELEGA E ACTIVATION BOX INNESTATE



- Si parla di auto-delega o chiamata interna quando un oggetto invia un messaggio a se stesso (es.: chiamata a un proprio metodo)
- L'auto-delega genera un'attivazione innestata

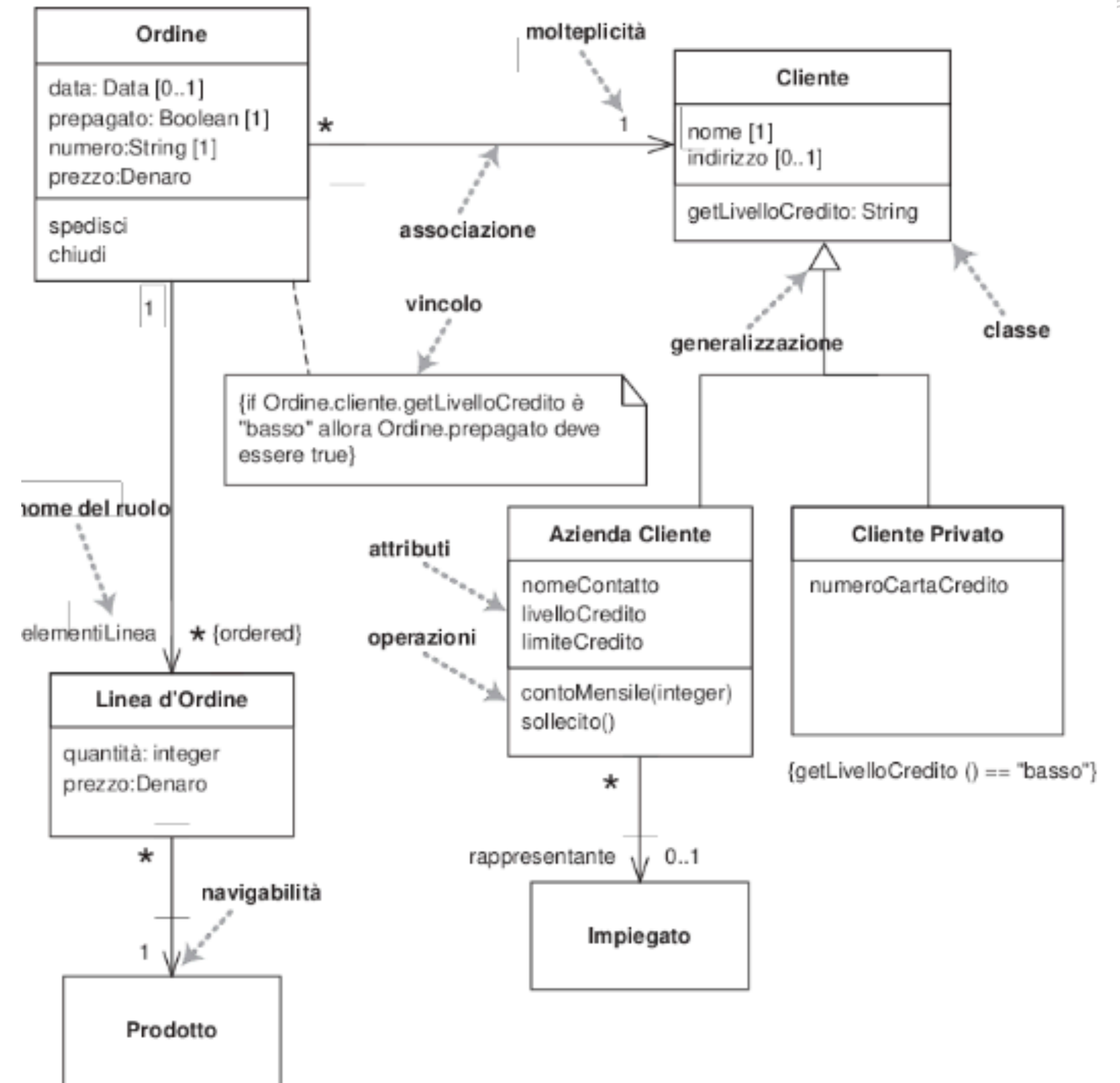




DAGLI SCENARI AI DIAGRAMMI DI SEQUENZA AL CODICE 00

DIAGRAMMI DI SEQUENZA: ESEMPIO

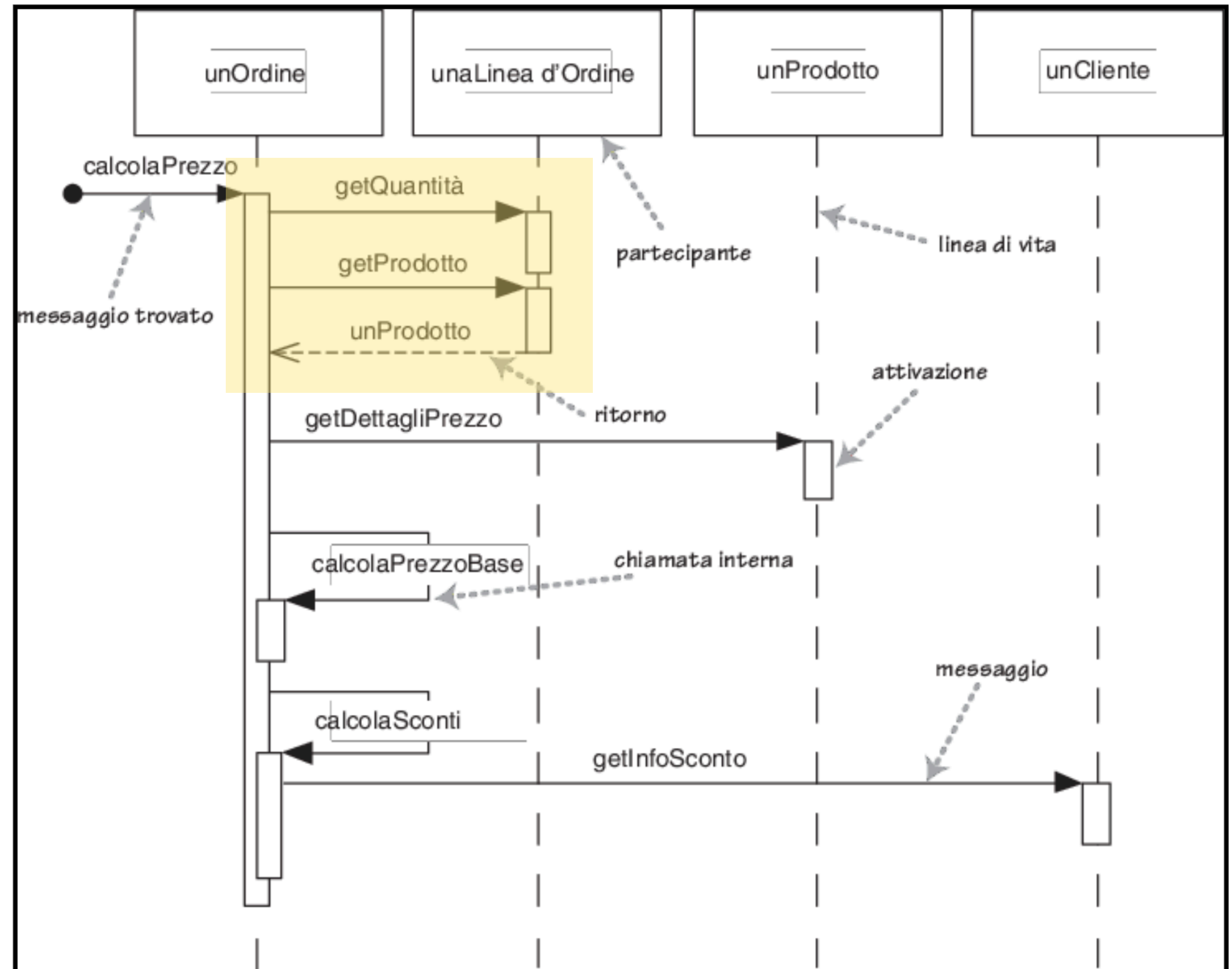
- Il sequence diagram può aiutare l'analista a perfezionare il diagramma delle classi, individuando le operazioni necessarie
- Consideriamo il caso d'uso **Calcola prezzo ordine**, ove il prezzo è determinato dal tipo di prodotto e dalla sua quantità



DIAGRAMMI DI SEQUENZA: ESEMPIO



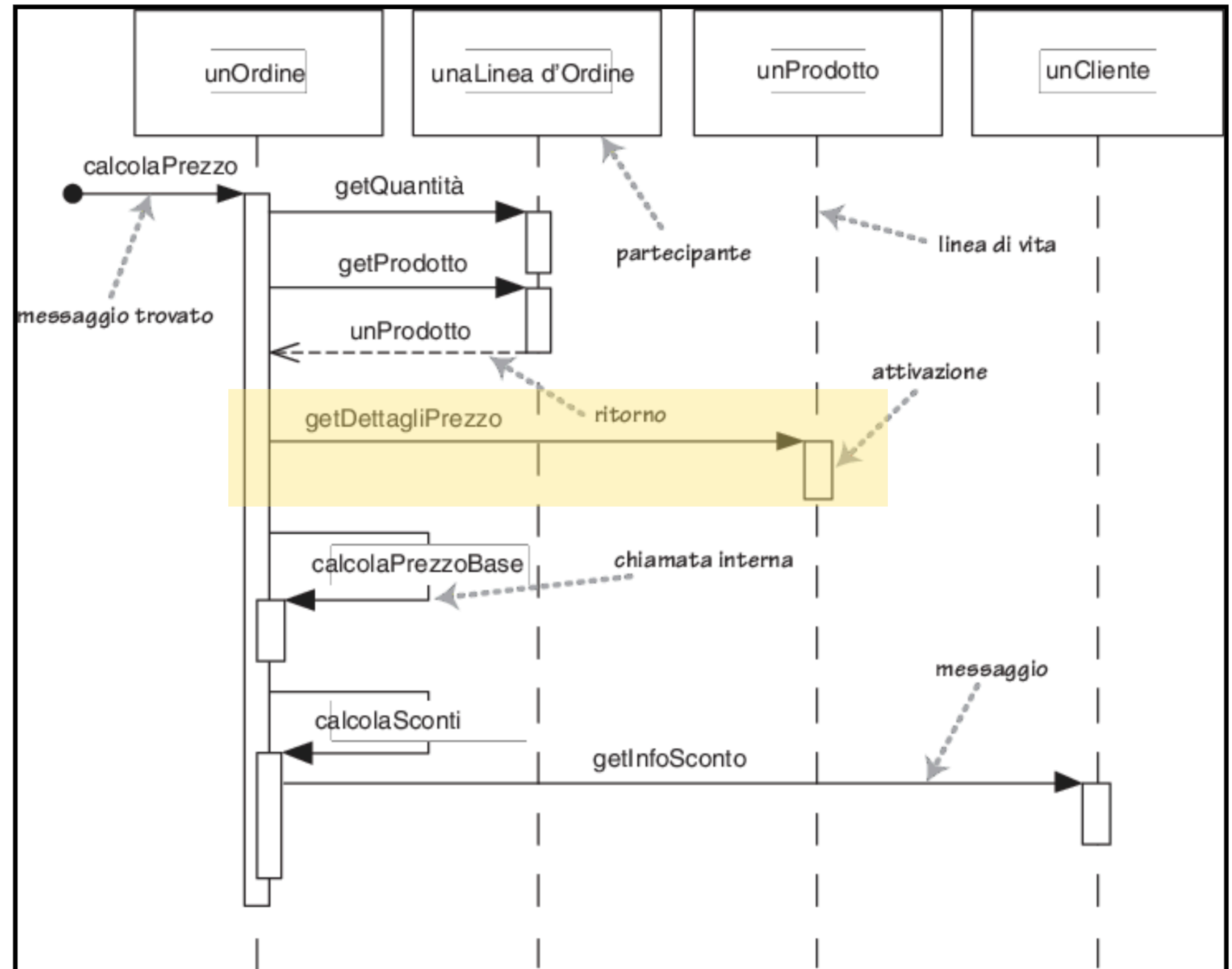
1. **L'ordine ottiene la quantità degli elementi delle linee, ossia i prodotti**
2. L'ordine deve ottenere il prezzo dei prodotti considerati
3. Viene calcolato il prezzo base dell'ordine
4. L'ordine calcola lo sconto globale, che dipende da informazioni sul cliente



DIAGRAMMI DI SEQUENZA: ESEMPIO



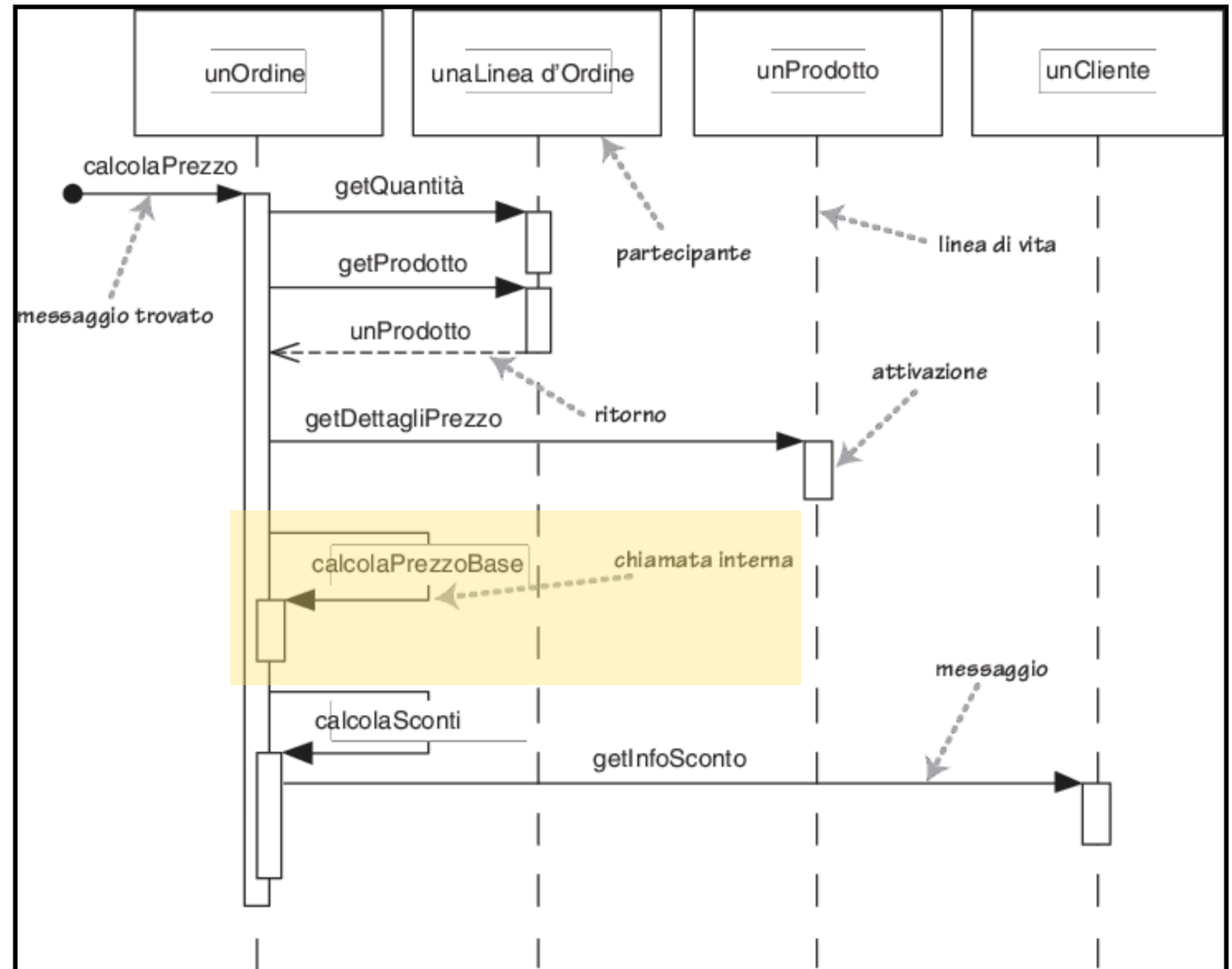
1. L'ordine ottiene la quantità degli elementi delle linee, ossia i prodotti
- 2. L'ordine deve ottenere il prezzo dei prodotti considerati**
3. Viene calcolato il prezzo base dell'ordine
4. L'ordine calcola lo sconto globale, che dipende da informazioni sul cliente



DIAGRAMMI DI SEQUENZA: ESEMPIO



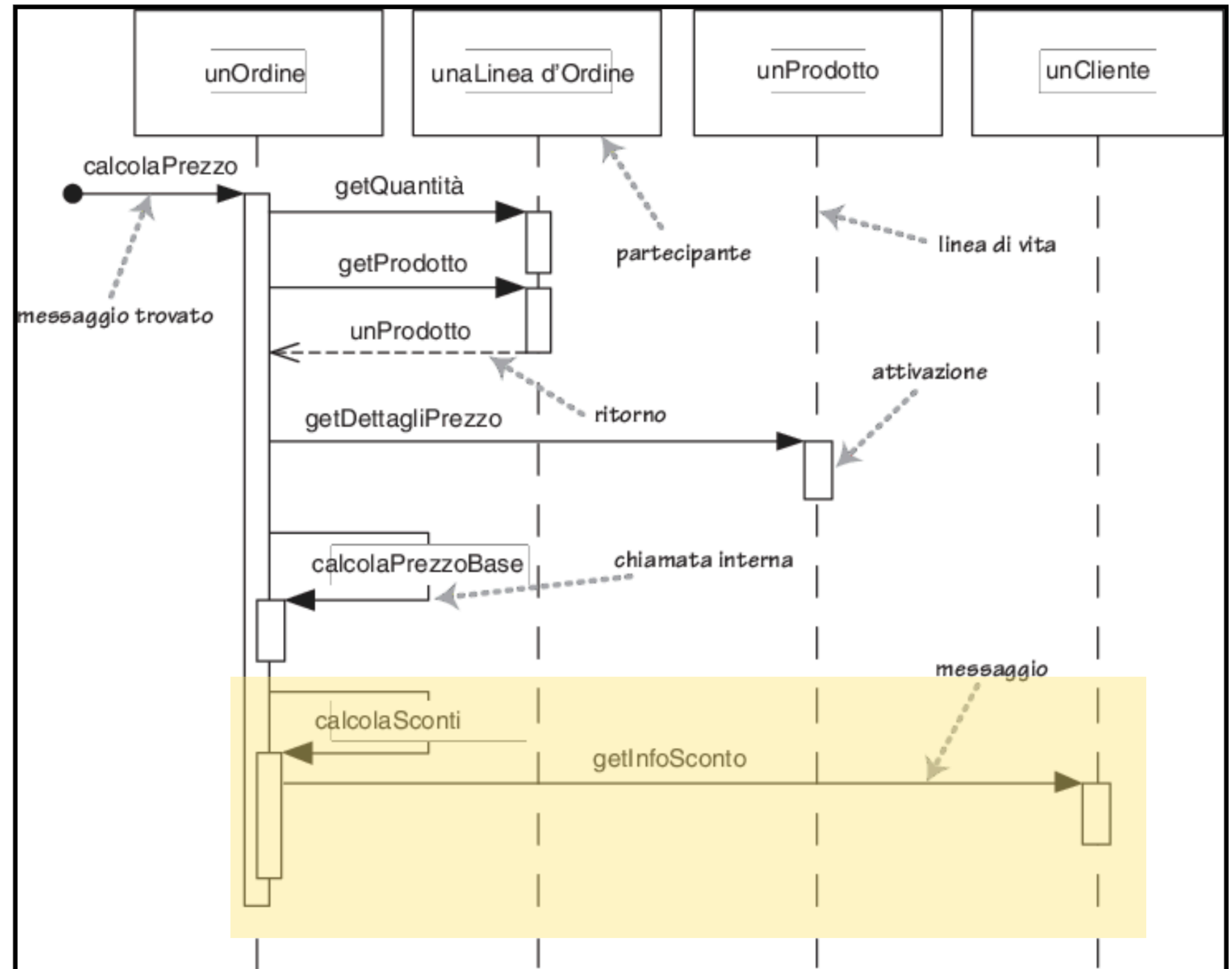
1. L'ordine ottiene la quantità degli elementi delle linee, ossia i prodotti
2. L'ordine deve ottenere il prezzo dei prodotti considerati
3. **Viene calcolato il prezzo base dell'ordine**
4. L'ordine calcola lo sconto globale, che dipende da informazioni sul cliente



DIAGRAMMI DI SEQUENZA: ESEMPIO



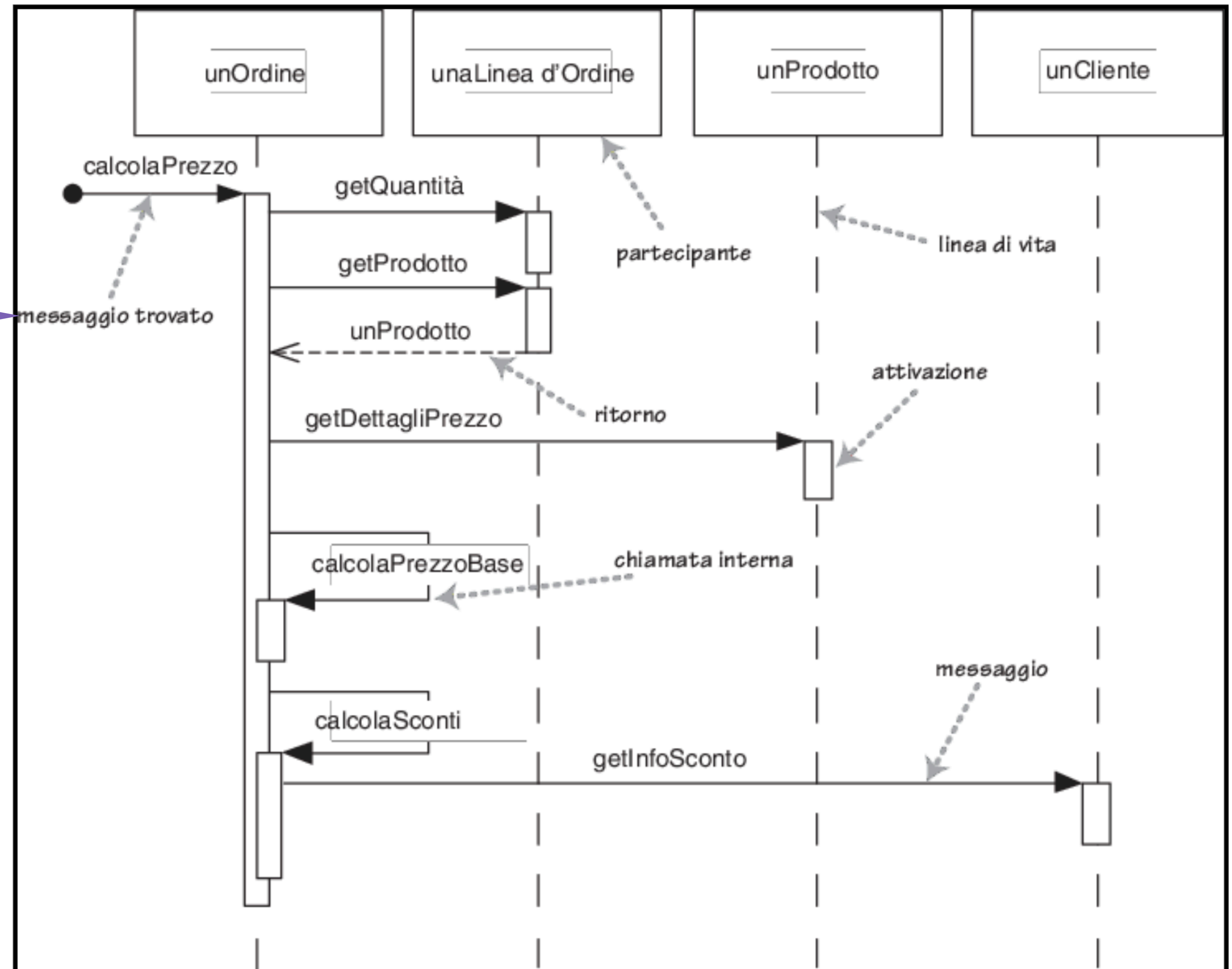
1. L'ordine ottiene la quantità degli elementi delle linee, ossia i prodotti
2. L'ordine deve ottenere il prezzo dei prodotti considerati
3. Viene calcolato il prezzo base dell'ordine
4. **L'ordine calcola lo sconto globale, che dipende da informazioni sul cliente**



DIAGRAMMI DI SEQUENZA: ESEMPIO



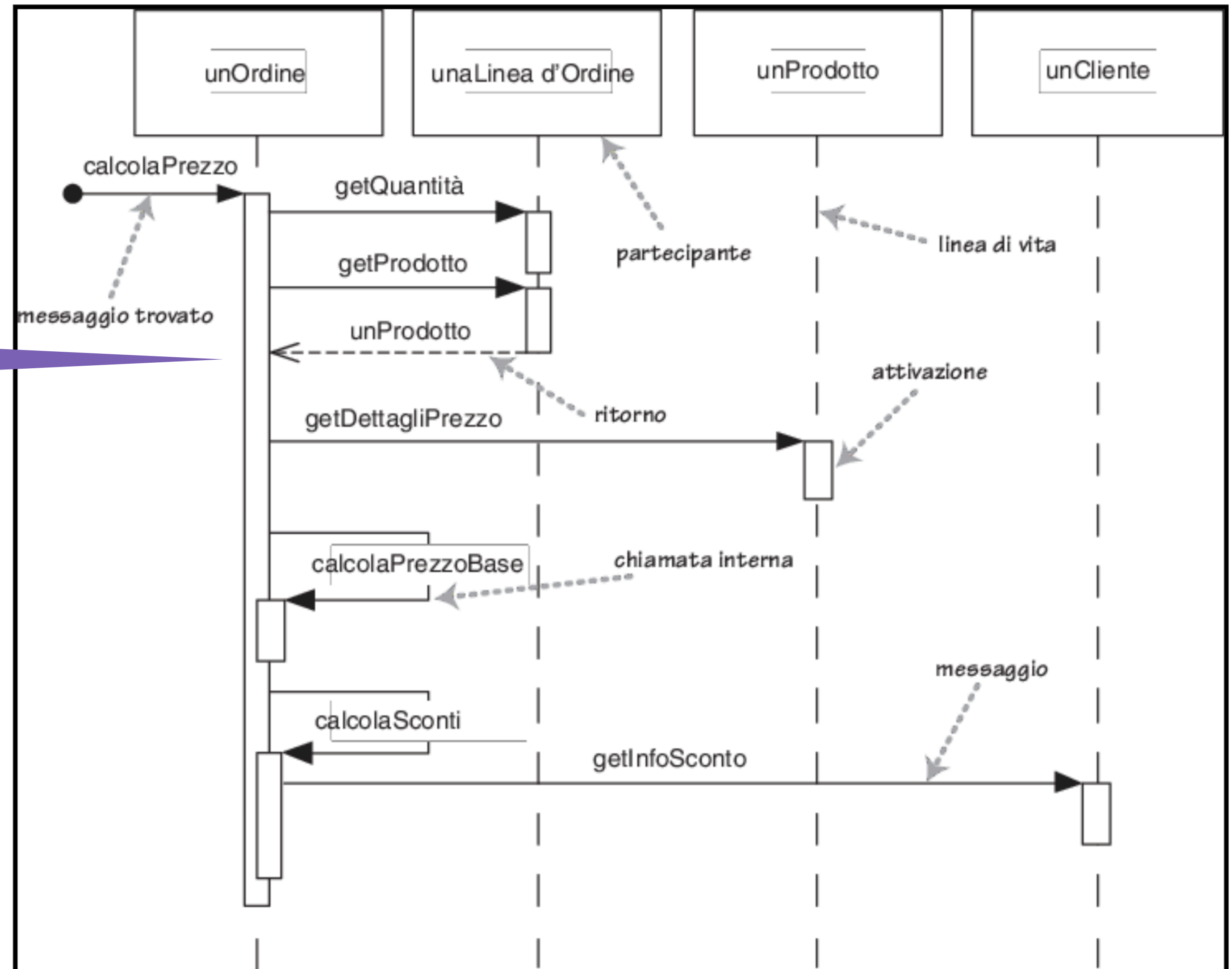
IL PRIMO MESSAGGIO NON SCATURISCE DA UN PARTECIPANTE AL DIAGRAMMA MA DA UNA FONTE ESTERNA INDETERMINATA, PERTANTO SI CHIAMA “MESSAGGIO TROVATO”



DIAGRAMMI DI SEQUENZA: ESEMPIO



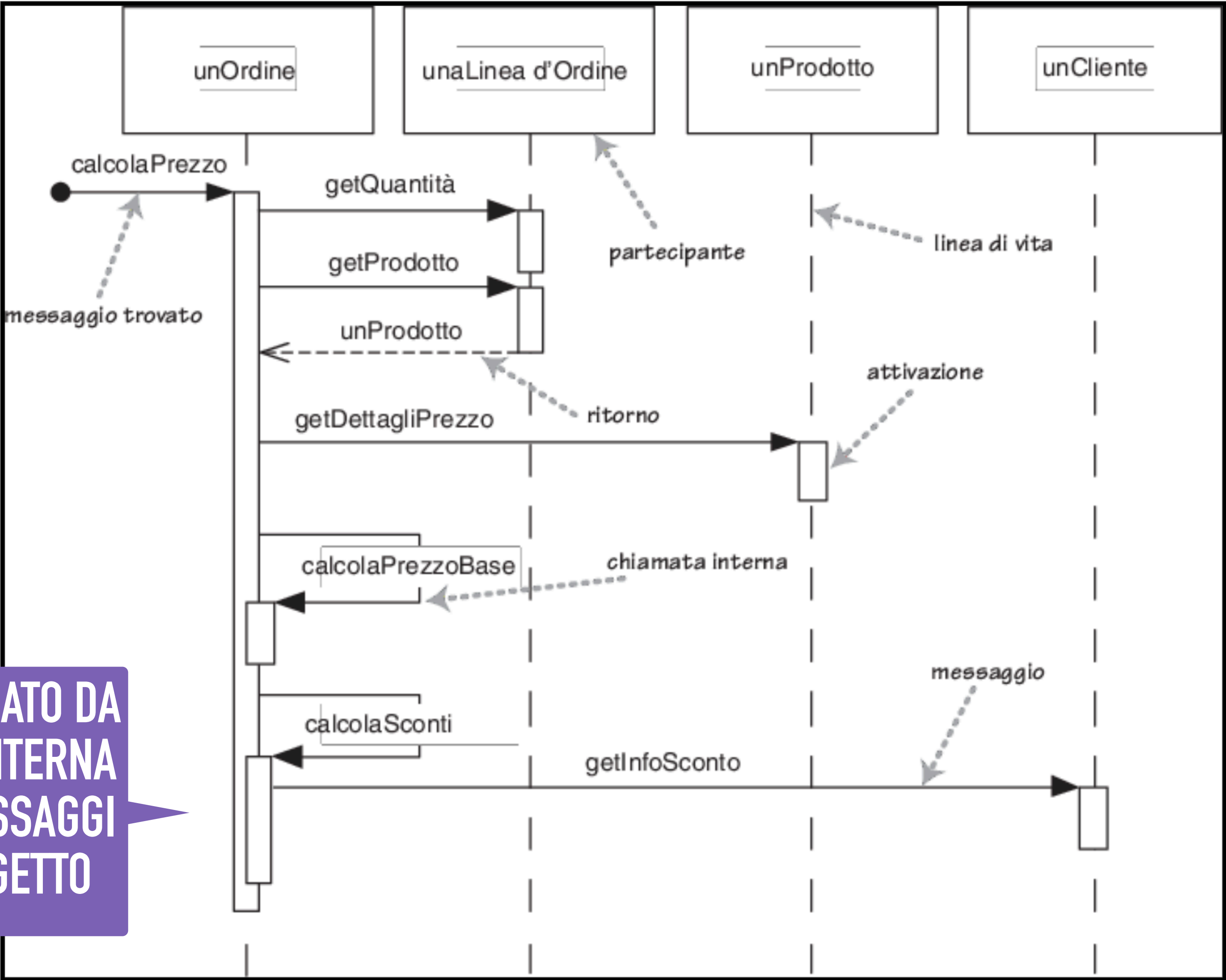
NELLA PRATICA, NON SEMPRE I MESSAGGI DI RITORNO SONO ESPlicitATI PER RENDERE PIÙ LEGGIBILE IL DIAGRAMMA. IN QUESTO CASO, L'AUTORE VOLEVA ESPlicitARE CHE GETDETTAGLIPREZZO VIENE CHIAMATO ESATTAMENTE SULLO STESSO PRODOTTO RESTITUITO DALLA LINEA D'ORDINE



DIAGRAMMI DI SEQUENZA: ESEMPIO



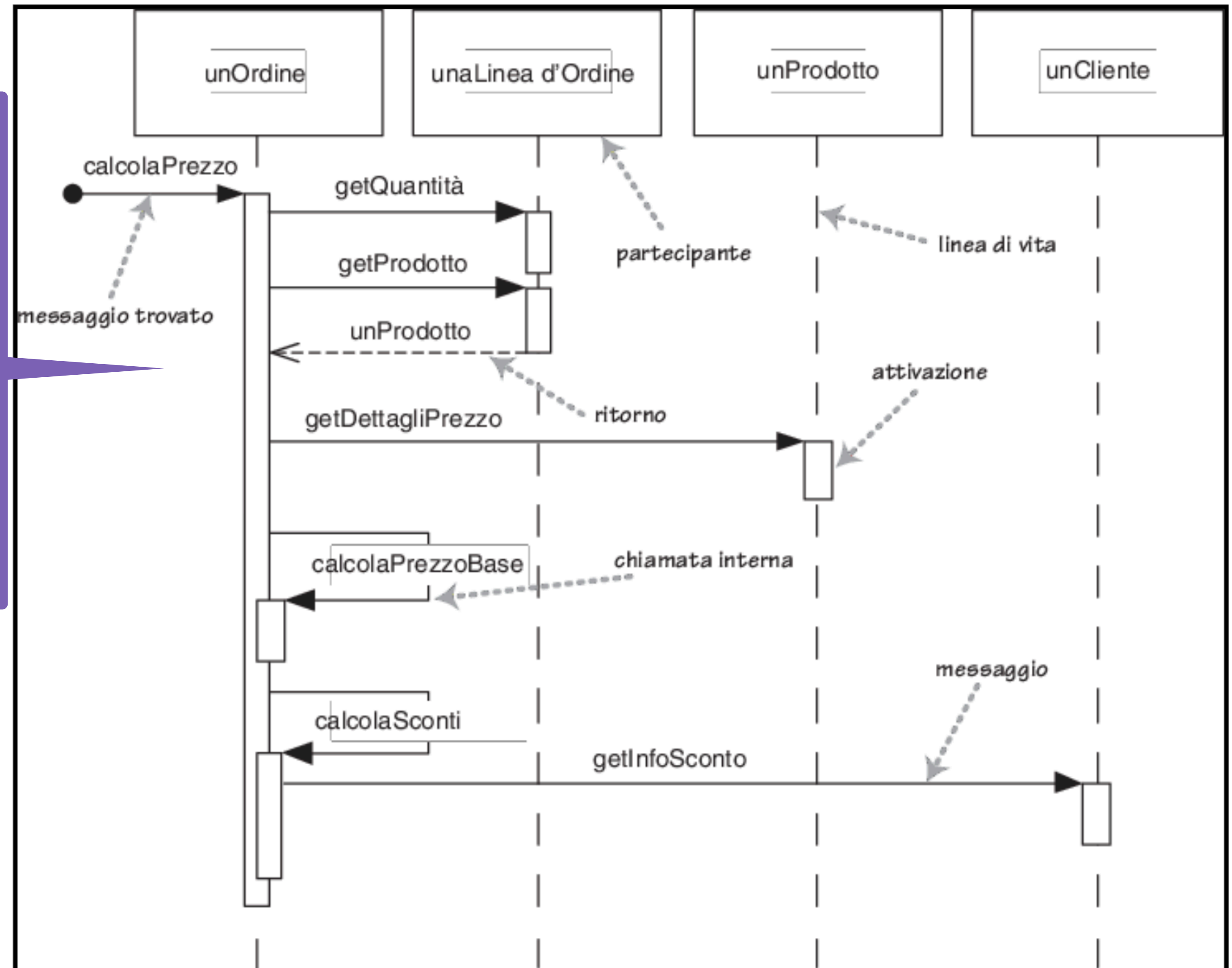
IL METODO INVOCATO DA
UNA CHIAMATA INTERNA
PUÒ INVIARE MESSAGGI
A UN ALTRO OGGETTO



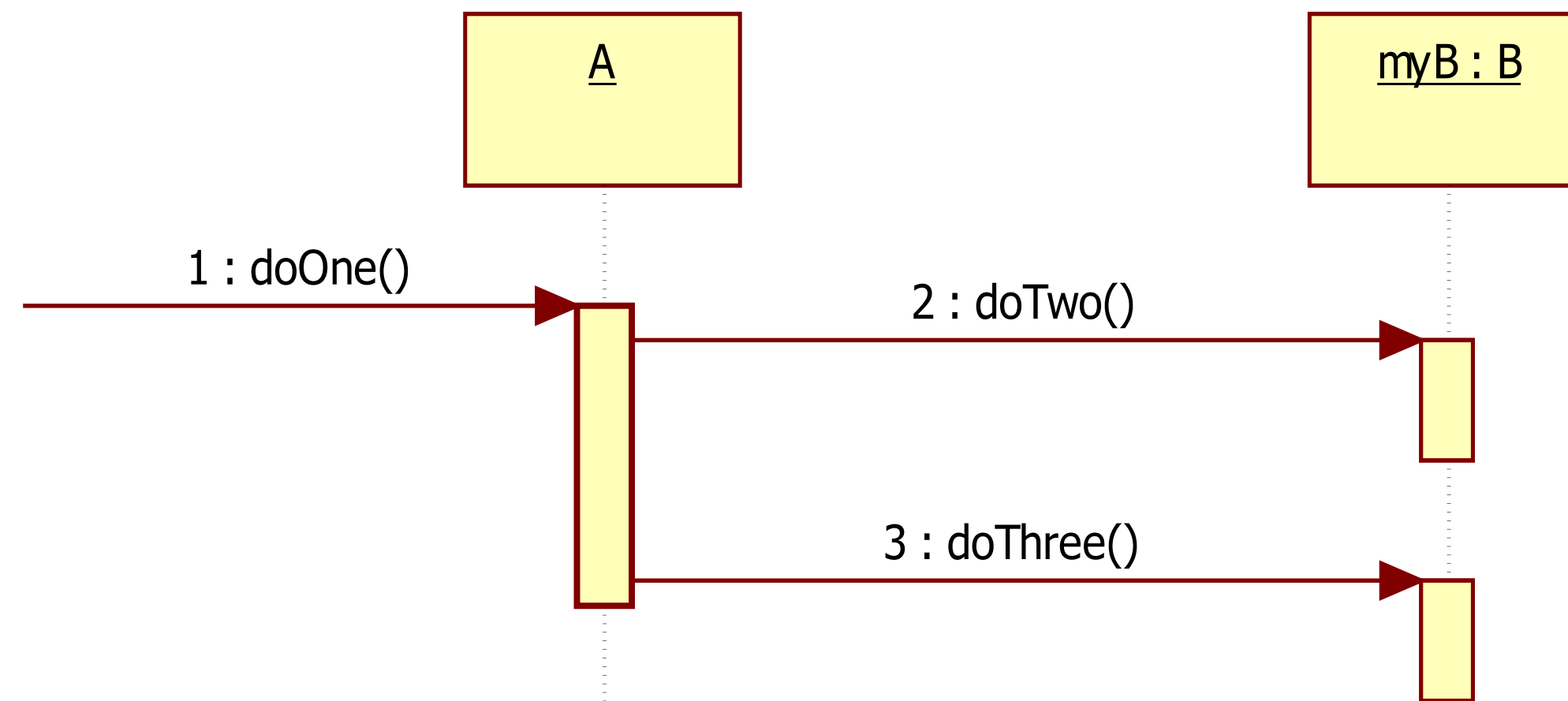
DIAGRAMMI DI SEQUENZA: ESEMPIO



QUESTA NOTAZIONE BASE NON É
SUFFICIENTE A RIPORTARE
L'INFORMAZIONE CHE I MESSAGGI VANNO
MANDATI A TUTTE LE LINEE D'ORDINE E A
TUTTI I PRODOTTI CHE FANNO PARTE
DELL'ORDINE.
PER QUESTO INTRODURREMO IN SEGUITO
ELEMENTI PIÙ AVANZATI DELLA NOTAZIONE



DIAGRAMMI DI SEQUENZA: ESEMPIO DI IMPLEMENTAZIONE



La classe A ha un attributo di tipo B ed un metodo doOne()

La classe B ha due metodi: doTwo() e doThree()

```
public class A
{
    private B myB = new B();
    public void doOne()
    {
        myB.doTwo();
        myB.doThree();
    }
    // ...
}
```



PARADIGMI DI MODELLAZIONE CENTRALIZZATI E DISTRIBUITI

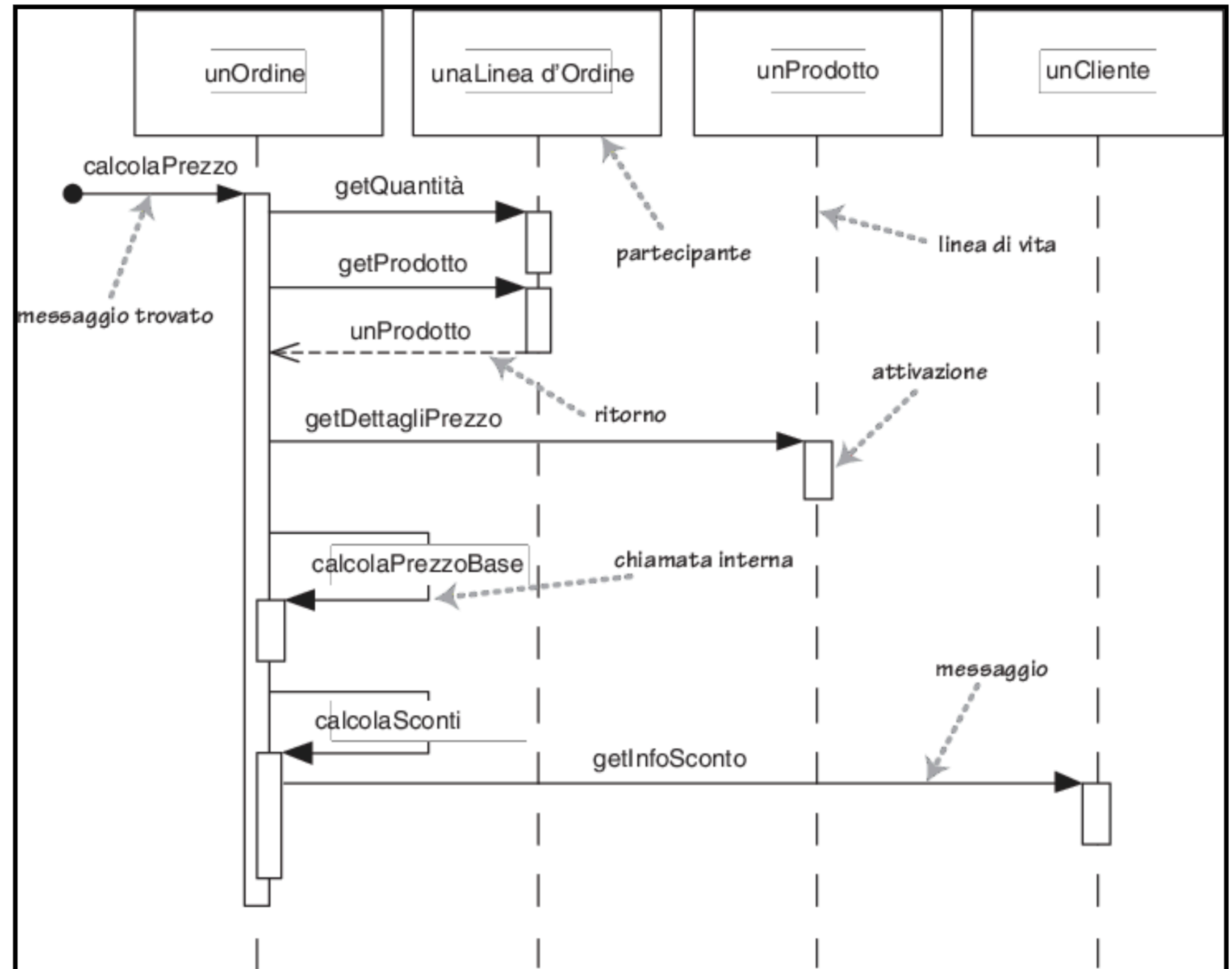
PARADIGMI DI MODELLAZIONE



- Due diversi paradigmi di modellazione in base a come viene gestito il controllo

- **Centralizzato: un singolo oggetto ha il controllo; gli altri forniscono servizi**

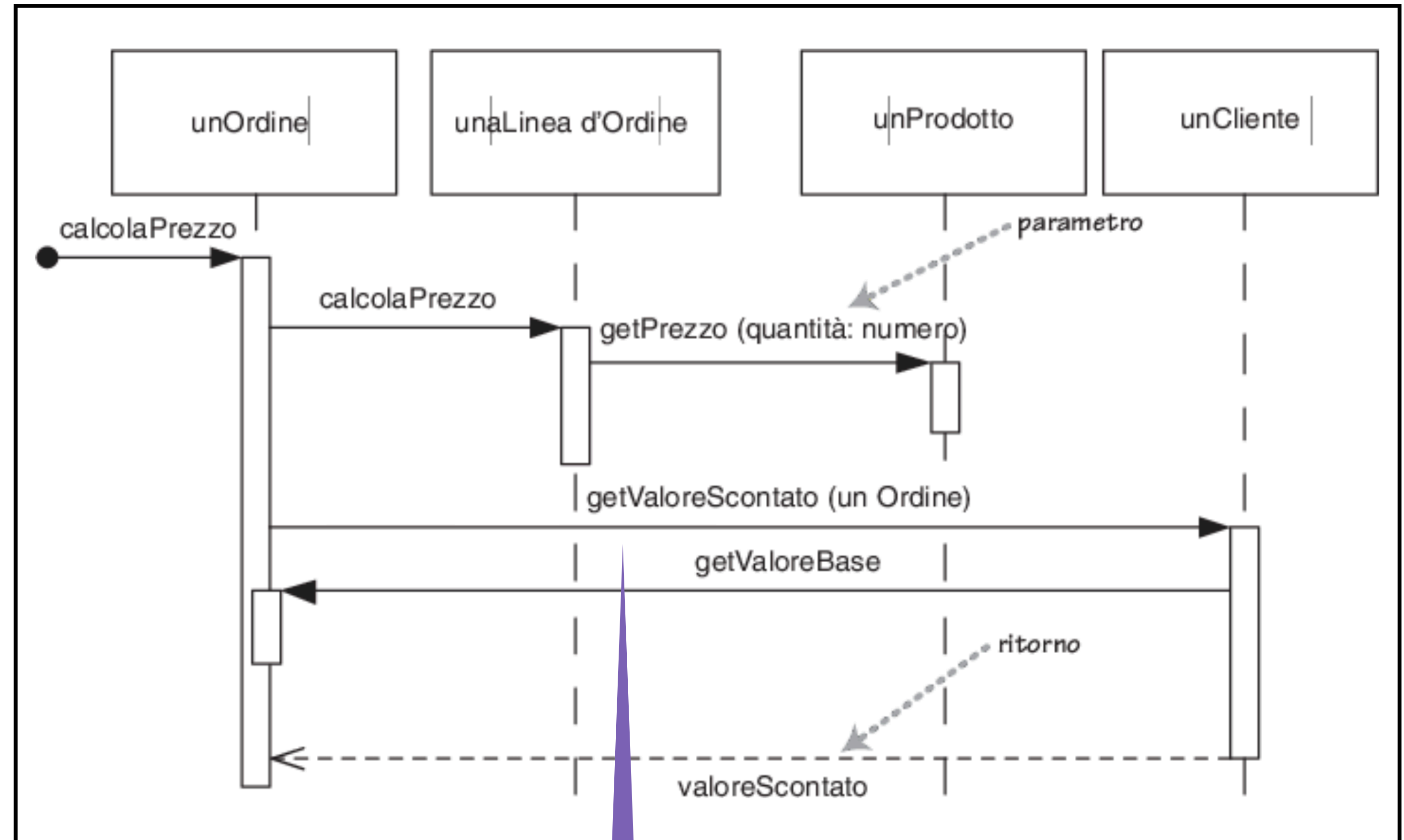
- Distribuito: le responsabilità sono distribuite tra gli oggetti partecipanti



PARADIGMI DI MODELLAZIONE



- Due diversi paradigmi di modellazione in base a come viene gestito il controllo
 - Centralizzato: un singolo oggetto ha il controllo; gli altri forniscono servizi
 - **Distribuito: le responsabilità sono distribuite tra gli oggetti partecipanti**



IN QUESTO CASO ABBIAMO MODELLATO ESPPLICITAMENTE ANCHE UN PARAMETRO



▸ **Controllo Centralizzato**

- ◎ **Pro:** Semplice soluzione che tende a raccogliere in un solo punto tutta l'elaborazione
- ◎ **Contro:** La logica che gestisce alcuni attributi si potrebbe trovare in classi diverse, rendendo non intuitiva la manutenzione del sistema

▸ **Controllo Distribuito**

- ◎ **Pro:** Conforme al paradigma OO; facilita la manutenzione raccogliendo nello stesso oggetto dati e logica che potrebbero dover essere modificati insieme; permette di usare il polimorfismo invece della logica condizionale
- ◎ **Contro:** Può risultare più dispersivo e difficile da leggere in fase di analisi



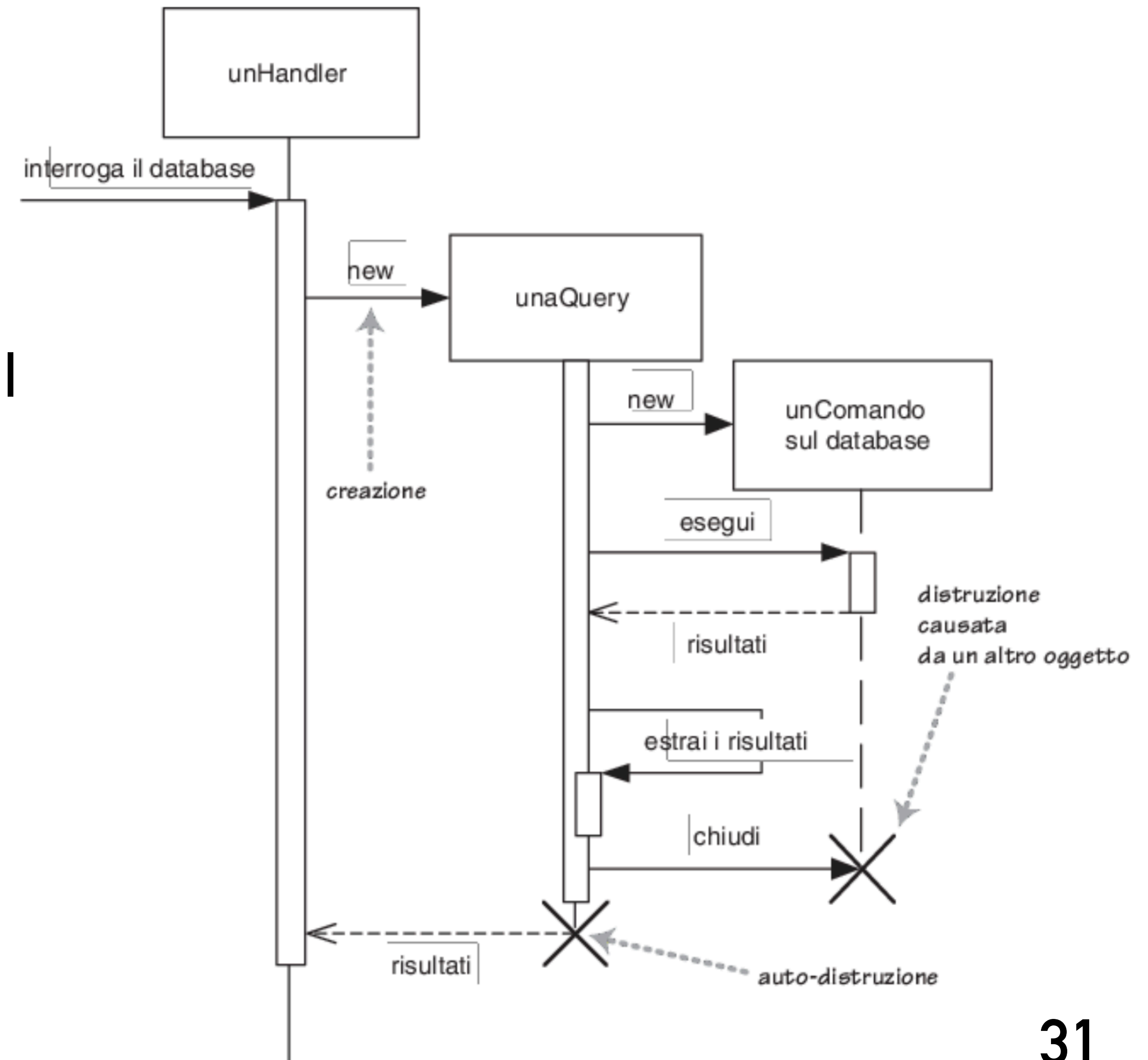
CREAZIONE E DISTRUZIONE DEGLI OGGETTI

CREAZIONE E DISTRUZIONE DI OGGETTI



► Creazione:

- Un messaggio al costruttore di un oggetto può creare un oggetto
- L'oggetto creato viene disegnato al termine della freccia corrispondente al messaggio
- Il messaggio costruttore può essere etichettato con il nome **new**
- Oggetti staticamente definiti partono dalla cima del diagramma
- Se il partecipante fa qualcosa appena creato, la activation box si disegna attaccata al rettangolo

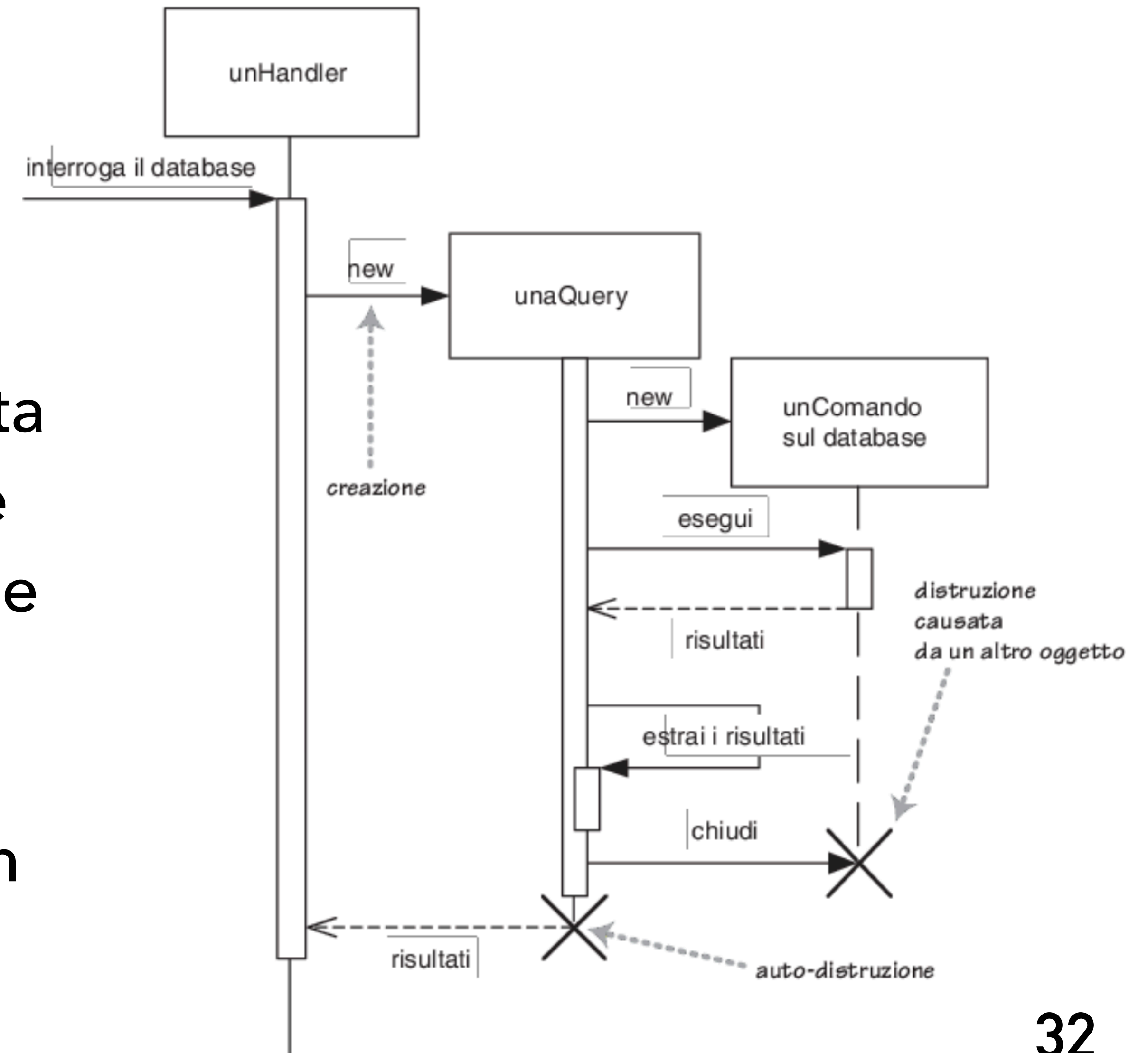


CREAZIONE E DISTRUZIONE DI OGGETTI



► Distruzione:

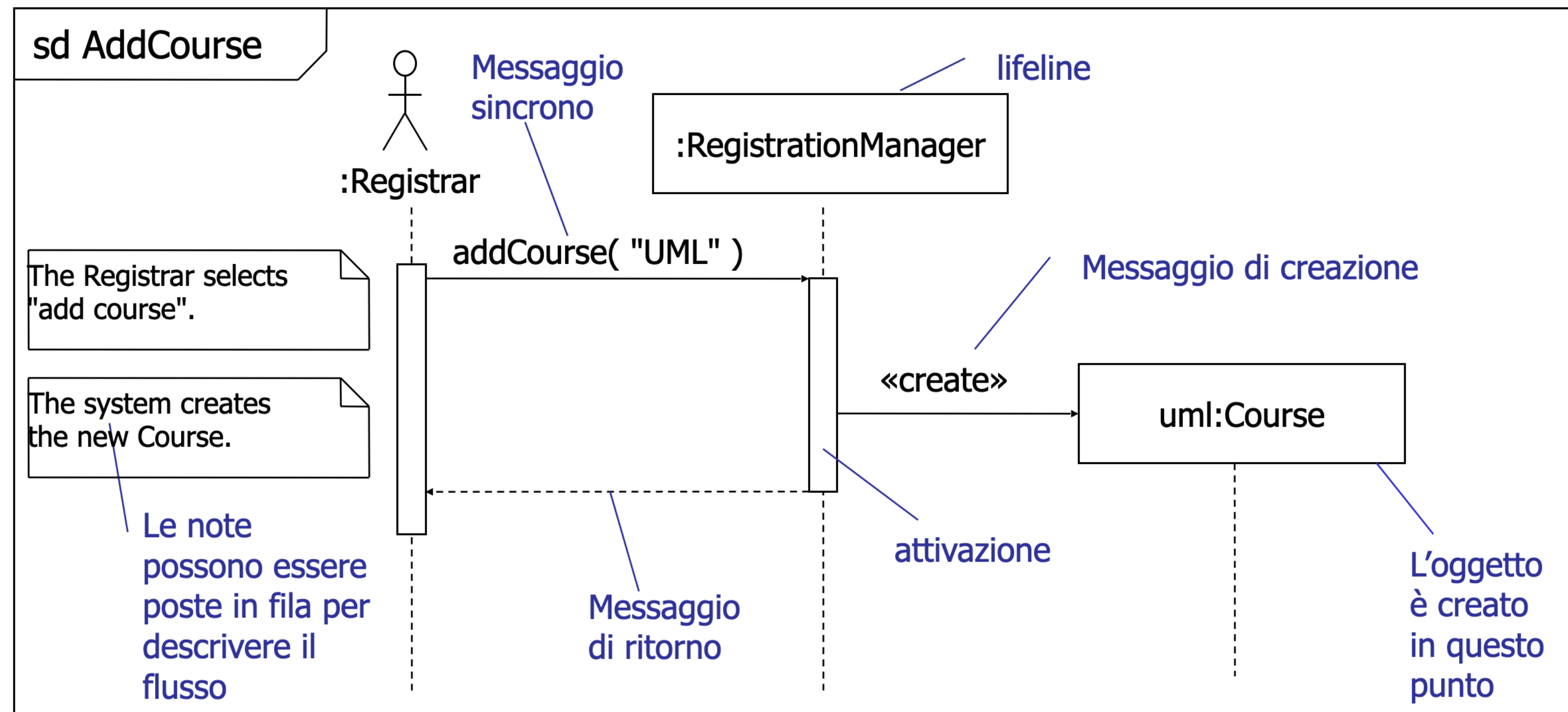
- Indicata con una X
- Se la freccia del messaggio tra oggetti termina con una X allora l'oggetto che invia il messaggio sta distruggendo quello che lo riceve
- Una X da sola alla fine della lifeline indica che l'oggetto sta distruggendo sé stesso
- Dopo la distruzione, l'oggetto non è più disponibile per l'elaborazione



MESSAGGI: RIEPILOGO



- Si possono notare le interazioni tra le lifeline come una sequenza ordinata temporalmente di eventi
- Gli activation box indicano quando un oggetto è coinvolto nel flusso di controllo dell'operazione



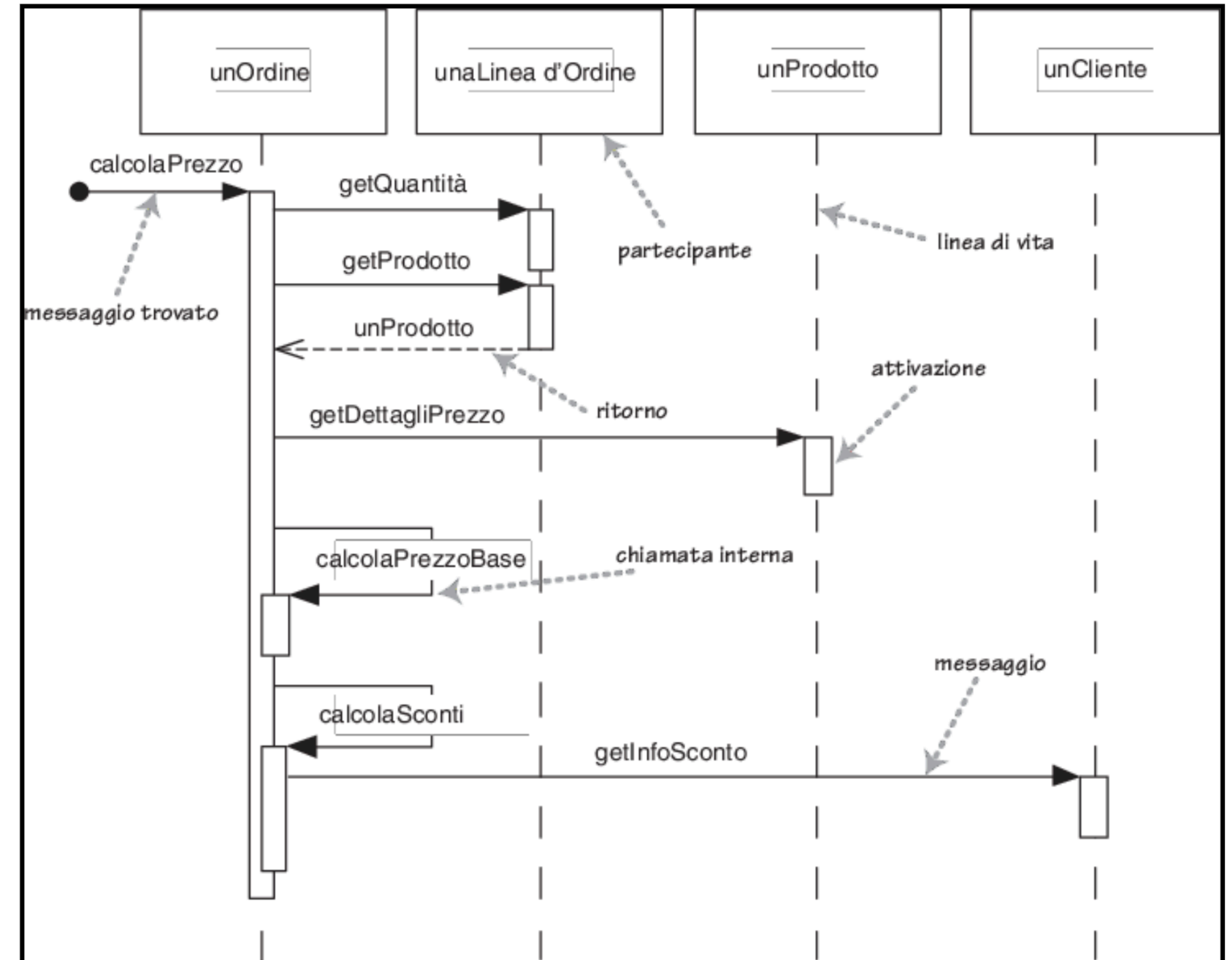


FRAME DI INTERAZIONE E FRAMMENTI COMBINATI

LIMITAZIONI DELLA NOTAZIONE BASE



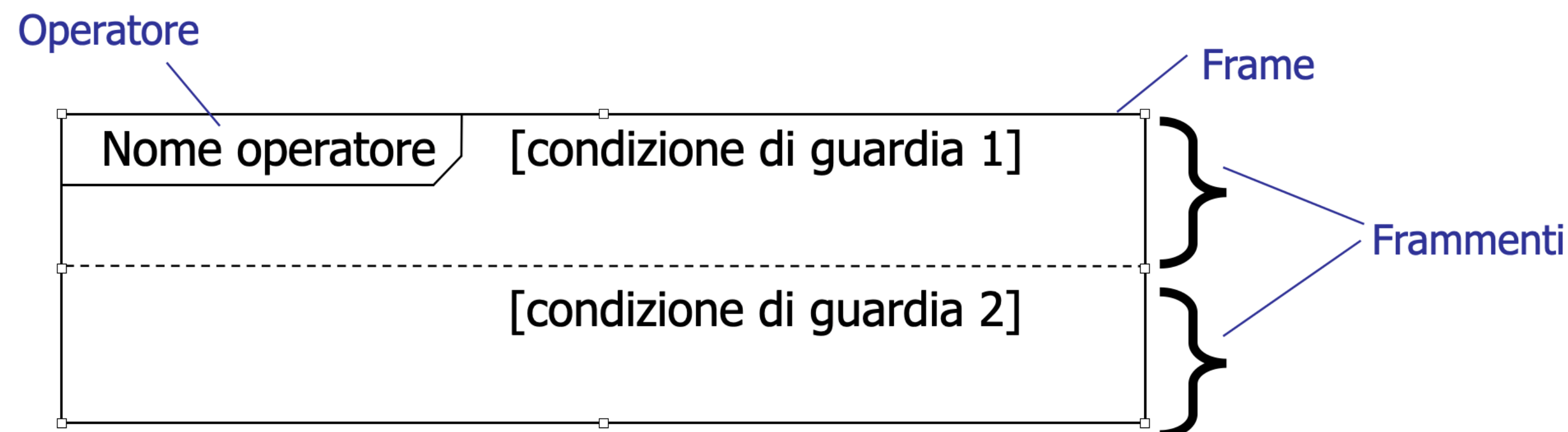
- ▶ Come abbiamo visto nell'esempio precedente, i diagrammi di sequenza sono più adatti a descrivere l'interazione tra oggetti che la logica di controllo
- ▶ La notazione base non permette di rappresentare comportamenti ciclici o condizionali
- ▶ Pertanto, in UML2 sono stati introdotti i frame ed i frammenti con i loro operatori



FRAME DI INTERAZIONE E FRAMMENTI COMBINATI



- I frame di interazione evidenziano una parte del diagramma e racchiudono una sottosequenza di messaggi
- Possono essere composti da uno o più frammenti combinati
- Ogni frame ha:
 - Un operatore, determina come vengono eseguiti i suoi operandi
 - Uno o più frammenti
 - Zero o più condizioni di guardia, stabiliscono se i frammenti corrispondenti devono essere eseguiti
- Ogni frammento può avere la propria guardia

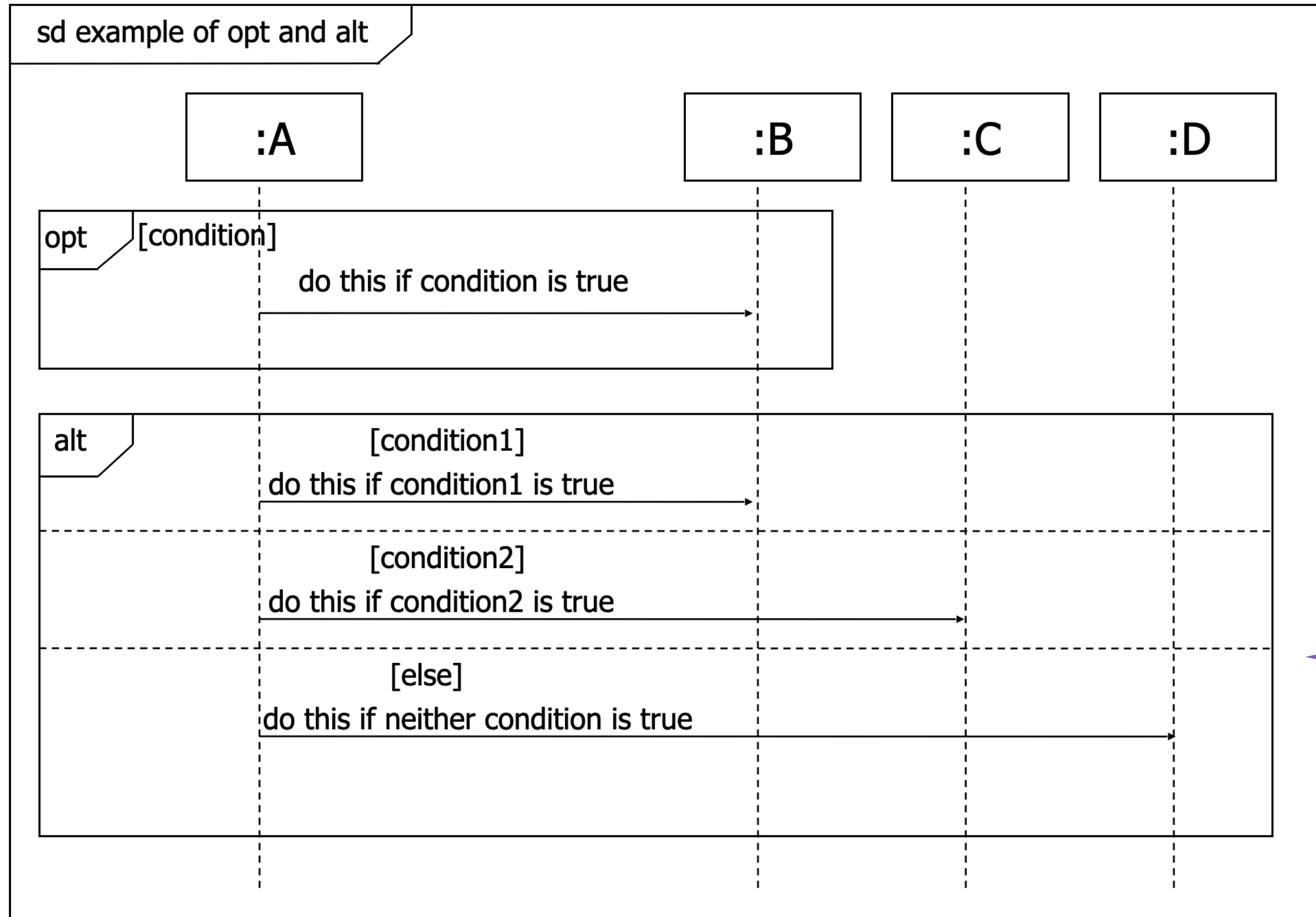


OPERATORI DI FRAMMENTO PIÙ USATI



Operatore	Nome completo	Semantica
opt	Optional	Il frammento corrispondente é eseguito solo se la sua condizione è vera. Costrutto if-then
alt	Alternatives	Molteplici frammenti in alternativa. Viene eseguito solo il frammento la cui condizione è vera. Ammette else come condizione di guardia. Costrutto if-then-else
loop	Loop	Ciclo. Il frammento é eseguito finché la condizione di guardia é verificata. La condizione é indicata all'inizio (while-do) o alla fine (do-while)
ref	Reference	Il frame rimanda ad un'interazione definita in un altro diagramma. Possono essere indicati dei parametri ed un tipo di ritorno. Nota: il frame deve racchiudere tutte le lifeline coinvolte nell'interazione

FRAMMENTI COMBINATI: OPT E ALT

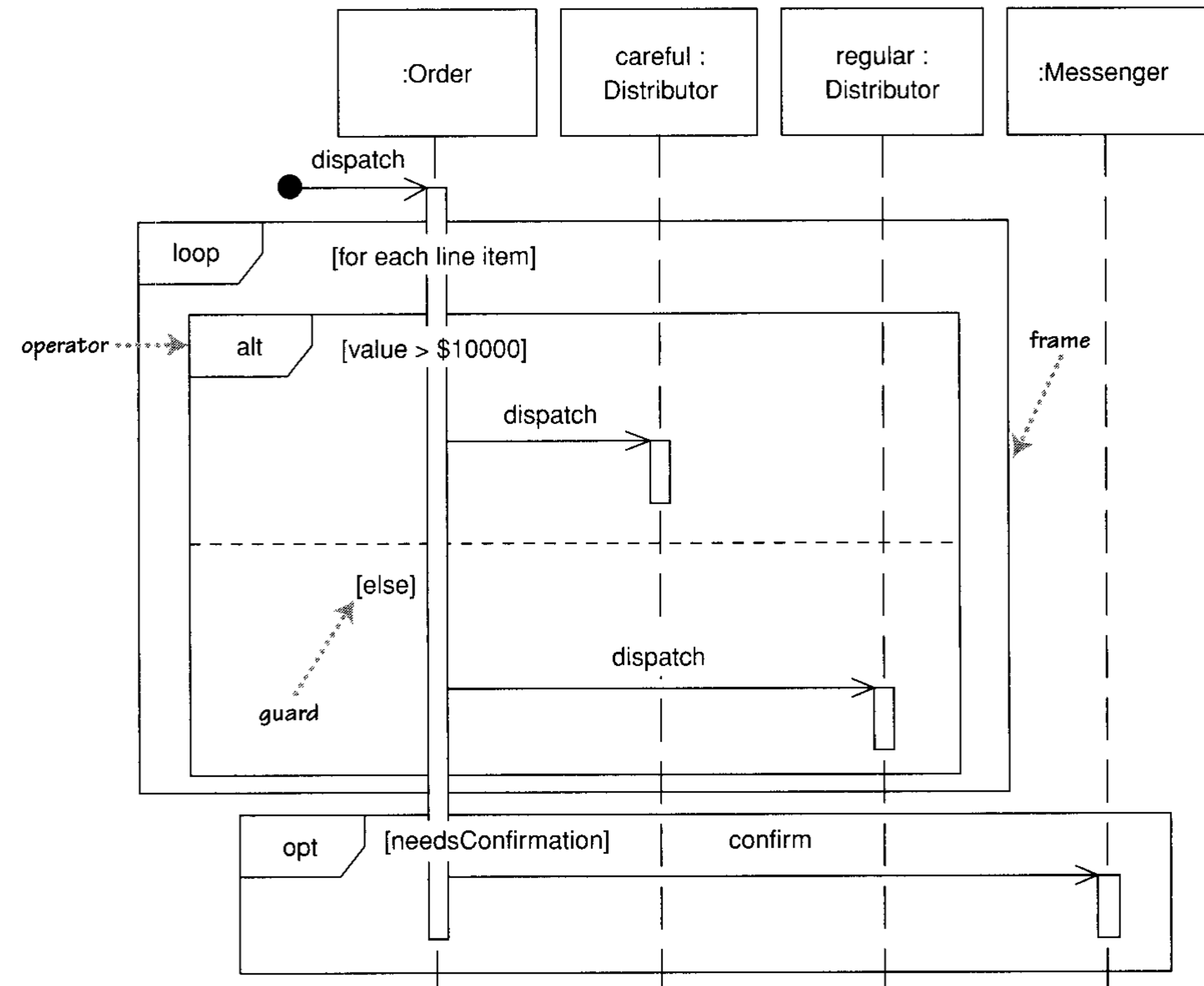


**I FRAMMENTI ALTERNATIVI
SONO SEPARATI DA UNA
LINEA TRATTEGGIATA**

ESEMPIO



```
procedure dispatch
  foreach (lineitem)
    if (product.value > $10K)
      careful.dispatch
    else
      regular.dispatch
    end if
  end for
  if (needsConfirmation) messenger.confirm
end procedure
```





► **Vantaggi**

- ⦿ Permettono di rappresentare costrutti condizionali e iterativi
- ⦿ Permettono di modellare sia lo scenario principale che quelli alternativi in un caso d'uso (altrimenti bisognerebbe disegnare un diagramma diverso per ogni scenario alternativo)

► **Svantaggi**

- ⦿ I frame appesantiscono il diagramma
- ⦿ Poco adatti a rappresentare algoritmi complessi (in questi casi sono preferiti activity e statechart diagrams)

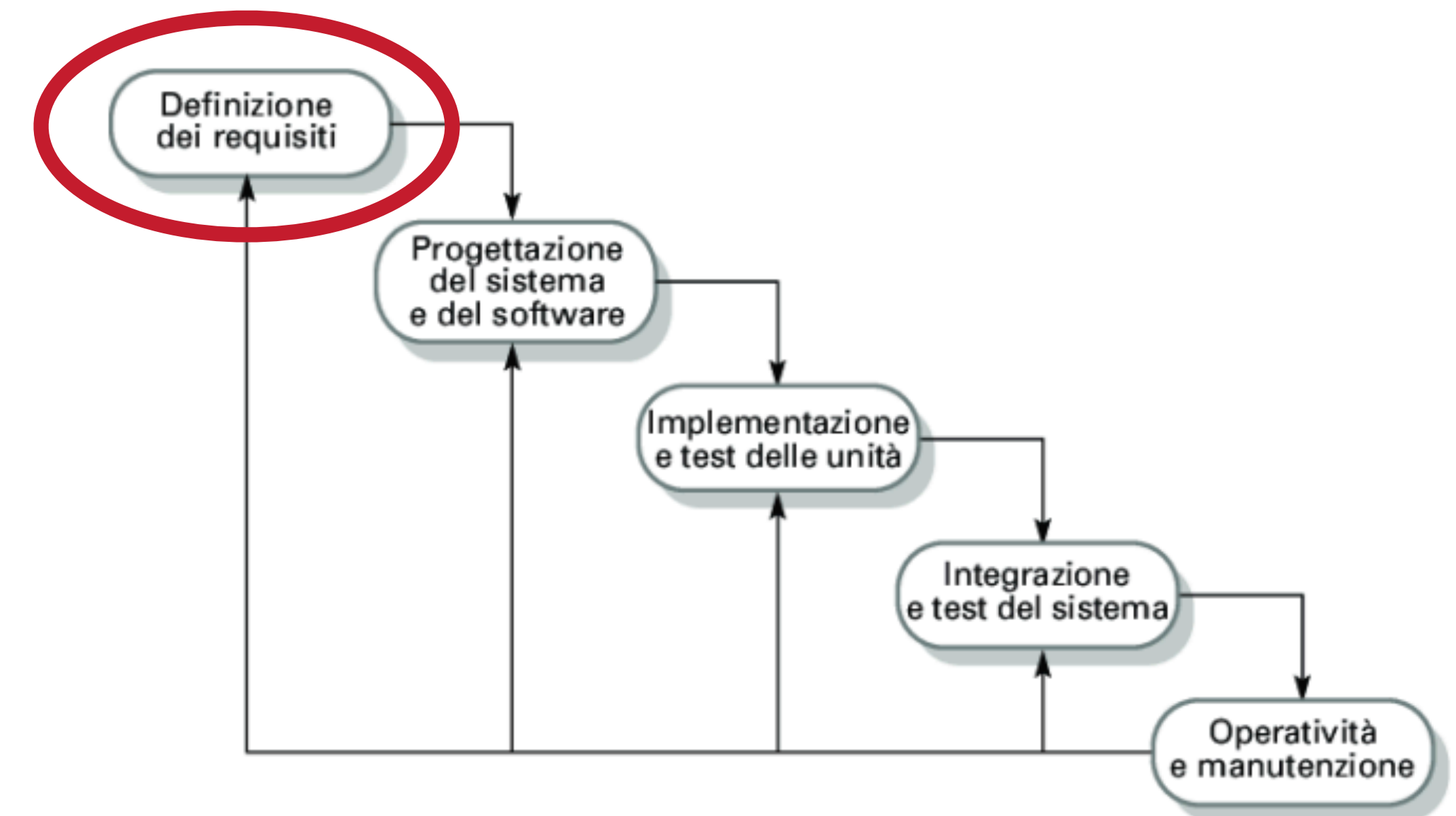


QUANDO UTILIZZARE I SEQUENCE DIAGRAM

SEQUENCE DIAGRAM NELLA FASE DI ANALISI DEI REQUISITI

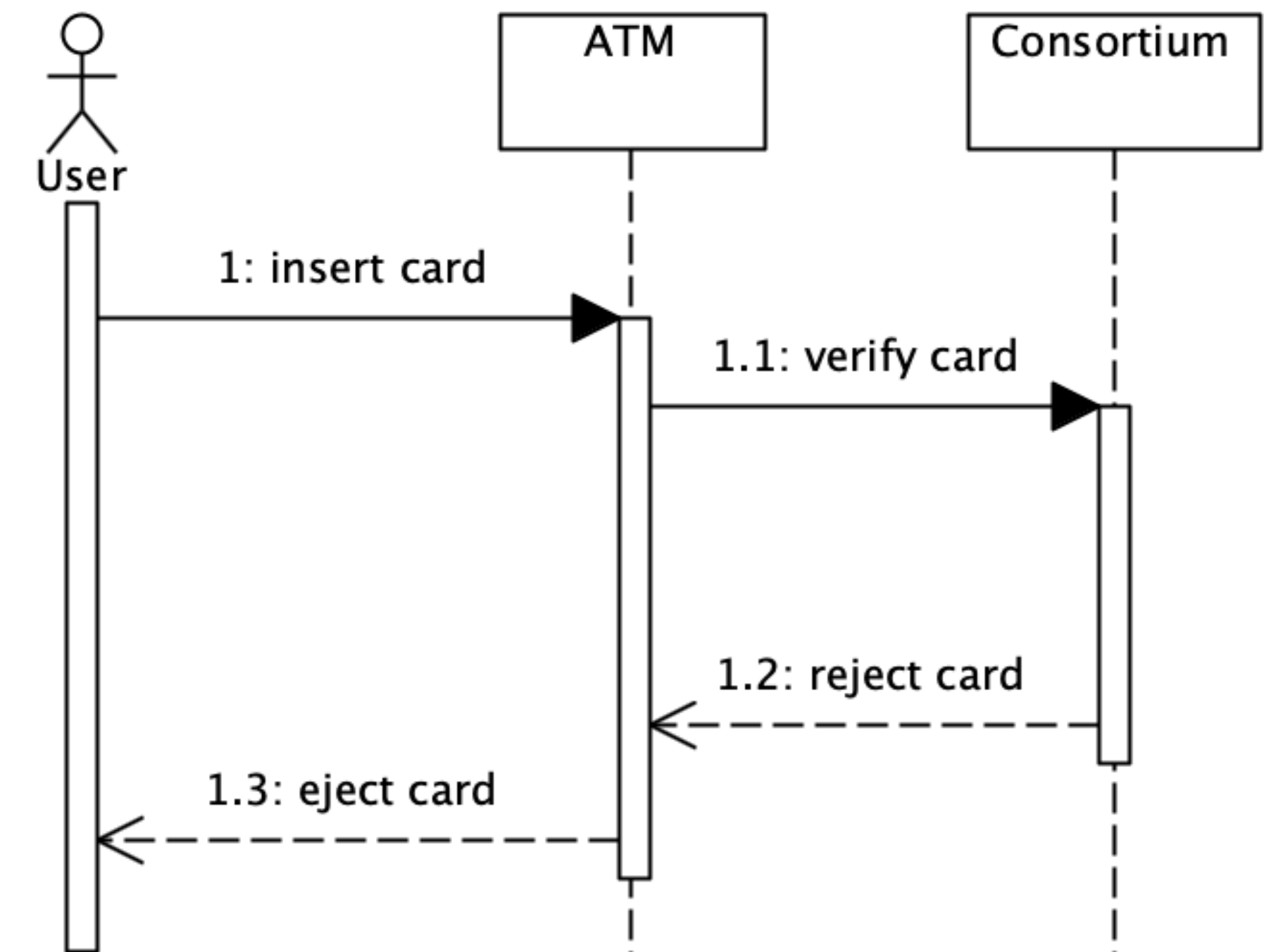


- I sequence diagram sono utilizzati in diverse fasi del ciclo di vita di un software
- In fase di analisi, un sequence diagram può essere una rappresentazione grafica di uno scenario di un caso d'uso



SEQUENCE DIAGRAM AD ALTO LIVELLO

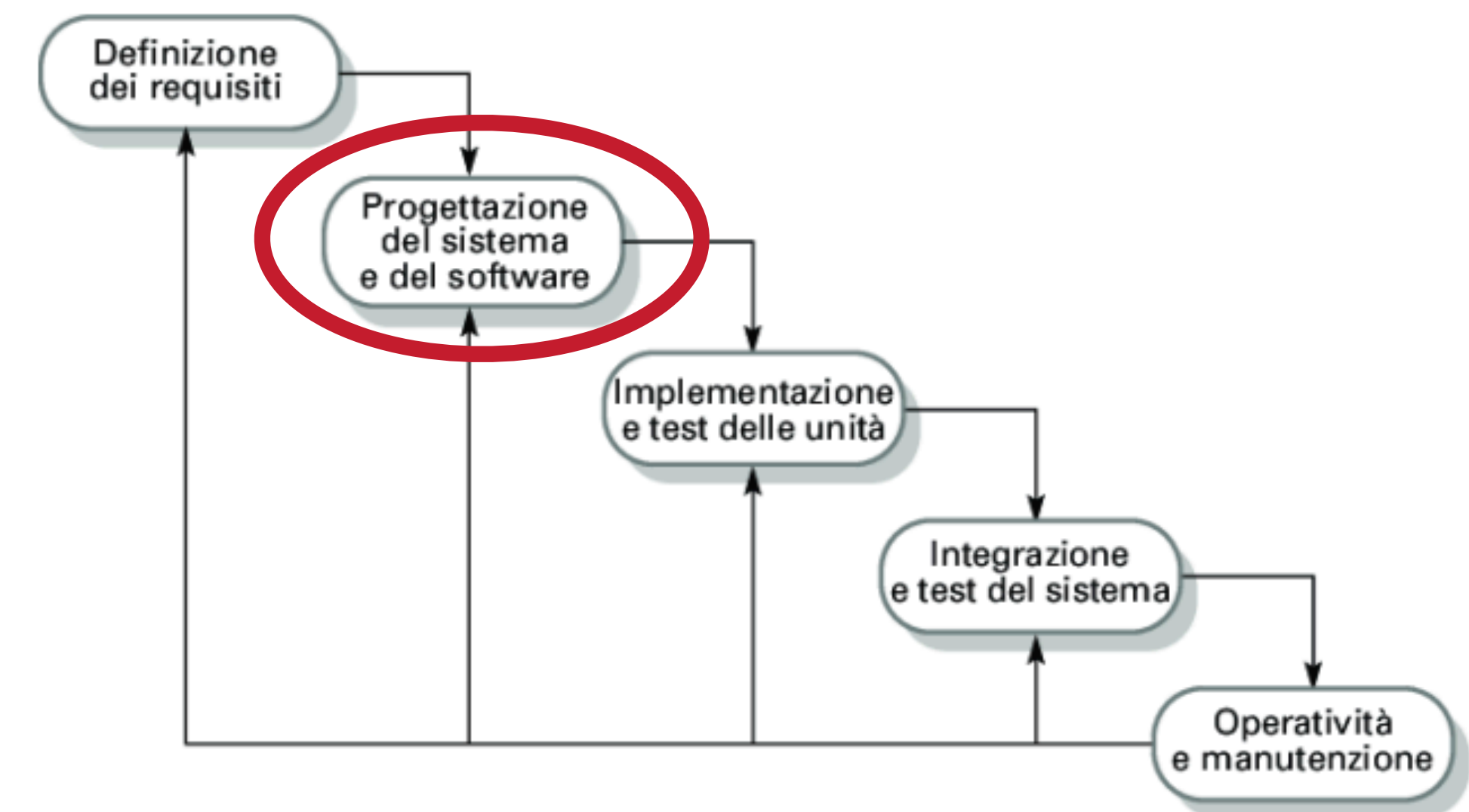
- In questo caso, il ruolo degli oggetti del sistema sarà ricoperto da un generico oggetto Sistema (ATM nell'esempio)
- Potranno comparire altri oggetti (indicanti ad esempio alcuni componenti architetturali interni o esterni quali server, database, etc.) se essi rappresentano attori o elementi vincolati nei requisiti



SEQUENCE DIAGRAM NELLA FASE DI PROGETTAZIONE



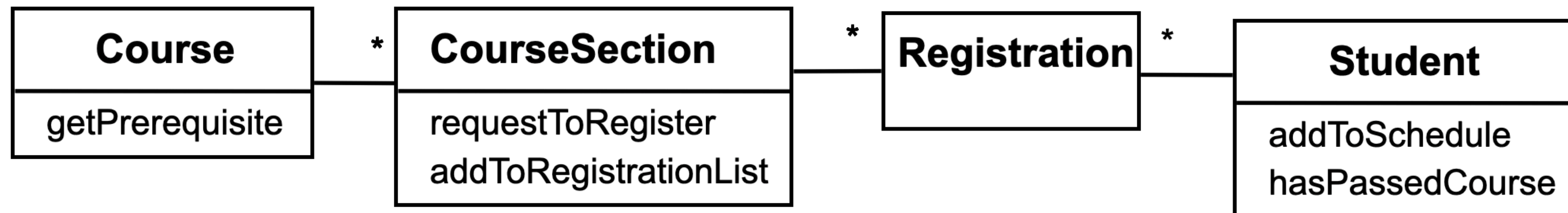
- In fase di progettazione i sequence diagram descrivono ancora scenari di casi d'uso
- I partecipanti sono istanze delle classi del sistema
- I messaggi sono le responsabilità o le operazioni stabilite dal diagramma delle classi
- Un diagramma di sequenza non dipende dal linguaggio di programmazione



SEQUENCE DIAGRAM NELLA FASE DI PROGETTAZIONE



- Vogliamo specificare la dinamica della funzionalità di Registrazione di uno studente ad una CourseSection
- Sequenza di operazioni necessarie:
 1. Invocare l'operazione di requestToRegister su una CourseSection
 2. Creare una istanza di Registration
 3. Collegare lo studente alla Registration
 4. Aggiungere la Registration alla RegistrationList di CourseSection



SEQUENCE DIAGRAM DETTAGLIATO

